



Guide de l'administrateur

Browser Policy Manager 0.9.3

Sommaire

Configuration requise et périmètre de livraison

- Configuration système minimale pour BPM
- Évaluer l'état de préparation actuel de la source à nœud unique
- Enregistrer les limites HA et de production différées

Déployer BPM depuis les sources sous Linux

- Préparer un hôte Linux pour le déploiement source
- Configurer l'extraction des sources Linux
- Configurer les paramètres d'exécution de la source Linux
- Vérifier un déploiement de source Linux
- Installer BPM depuis les sources sur Ubuntu 26.04 LTS
- Installer BPM depuis les sources sur Debian 13.5
- Installer BPM depuis les sources sur Fedora Linux 44
- Installer BPM à partir des sources sur Linux Mint 22.3
- Installer BPM depuis les sources sur la branche stable de Manjaro

Déployer BPM depuis les sources avec Windows WSL

- Préparer Windows 10/11 pour le déploiement de la source WSL
- Configurer la récupération de la source BPM dans WSL
- Configurer le runtime WSL et l'accès au navigateur
- Vérifier le déploiement de la source BPM via WSL

Exploiter et mettre à jour BPM depuis les sources

- Examiner les sources de configuration de DevOps
- Planifier le stockage, les journaux et les preuves de sauvegarde
- Examiner le réseau, CORS et les limites de sécurité
- Enregistrer les limites source-opération
- Préparer les éléments de preuve avant une mise à jour depuis les sources
- Actualiser la révision source et les dépendances
- Exécuter les migrations et les vérifications de mise à jour de la source
- Vérifier la mise à jour et appliquer les conditions d'arrêt de restauration
- Vérifier l'état et la préparation du BPM

Intégrer BPM par l'API

- Audience d'intégration et limite de l'API
- Modèles d'intégration pris en charge
- Conventions API

- Limites actuelles de l'API et contraintes de sécurité
- Réutiliser les exemples d'API et les modèles d'environnement DevOps
- Synchroniser les données du cycle de vie du profil
- Archiver, restaurer, supprimer définitivement ou réinitialiser les profils
- Importer Firefox policies.json via l'API
- Exporter un fichier Firefox policies.json canonique via l'API
- Valider un fichier Firefox policies.json candidat via l'API

Exécuter les flux de cycle de vie et rétablir le service

- Exécuter un scénario d'extraction, de comparaison et de mise à jour du produit de contrôle
- Exécuter un scénario d'importation, de révision, d'exportation et de transfert de conformité
- Extraire et lire les profils BPM à partir d'un produit de contrôle externe
- Valider et appliquer une mise à jour de profil avec expected_revision
- Exécuter le transfert des métadonnées d'importation, de révision, d'exportation et de conformité
- Démarrer le produit de contrôle après vérification de l'intégrité et de l'état de préparation de BPM
- Récupérer des échecs d'intégration et enregistrer les preuves d'audit

Résoudre les problèmes de BPM et évaluer la préparation à la production

- Résoudre les problèmes de démarrage échoué et les sondes inaccessibles
- Résoudre les erreurs de cache de schéma et de validation d'API
- Résoudre les échecs d'importation et d'exportation de Firefox
- Résoudre les problèmes de base de données et de stockage
- Dépanner le réseau WSL et les dépendances obsolètes
- Dépanner un portail de documentation indisponible
- Examiner les questions sur l'exposition du réseau et la préparation au proxy
- Planifier les entrées de surveillance, les preuves de sauvegarde et les fenêtres de mise à jour

Assistant de documentation BPM

- Utiliser l'assistant de documentation dans BPM 0.9.3
- Maintenir la limite de l'assistant de documentation dans BPM 0.9.3

Configuration requise et périmètre de livraison

Configuration système minimale pour BPM

Utilisez ces minimums pour le flux d'exécution BPM depuis les sources pris en charge dans la version 0.9.3.

Plateforme BPM de base

Exécutez BPM avec le flux depuis les sources Linux x86_64 pris en charge : Ubuntu 26.04 LTS, Debian 13.5, Fedora Linux 44, Linux Mint 22.3 ou la branche stable actuelle de Manjaro. Windows 10 et Windows 11 ne sont pris en charge que comme hôte WSL 2 pour ce flux Linux. BPM ne fournit ni service Windows natif ni programme d'installation. Prévoyez CPython 3.14, deux cœurs CPU, 4 Go de RAM et 4 Go d'espace disque inscriptible pour le checkout, l'environnement virtuel, la base de données, les journaux et la génération de la documentation.

Navigateur, réseau et accès

Utilisez un navigateur de bureau actuel avec JavaScript et HTTPS activés pour l'interface Web BPM et le portail de documentation. La liste des schémas Firefox pris en charge décrit la compatibilité des données de stratégie ; elle n'impose pas l'exécution de l'interface BPM dans Firefox. HTTPS sortant et sudo ne sont nécessaires que pour installer des paquets système ou des dépendances des sources. Le processus doit s'exécuter avec un utilisateur Linux ordinaire qui peut écrire dans le checkout BPM et les répertoires de données configurés ; n'exécutez pas l'application en tant que root uniquement pour l'exploiter.

Assistant de documentation dans 0.9.3

Le chat d'assistance documentaire est disponible comme interface de lecture, mais la version 0.9.3 ne fournit ni modèle local, ni index RAG, ni embeddings, ni réponses générées, ni citations, ni accès à des sources externes. Il n'ajoute donc aucune exigence de CPU, de RAM, de disque, de GPU, de NPU, de compte cloud ou de réseau au flux BPM de base.

Chaque question envoyée affiche l'avis localisé d'entraînement du modèle local. Utilisez la recherche déterministe indépendante dans la documentation pour les informations produites actuelles.

Évaluer l'état de préparation actuel de la source à nœud unique

Décidez si l'instance BPM exécutée à la source actuelle est prête pour un fonctionnement contrôlé à nœud unique et nommez les preuves qui soutiennent cette décision.

Le déploiement de Linux ou Windows 10/11 via la source WSL est terminé, les dépendances sont installées et les valeurs `BPM_DATABASE_URL`, `BPM_DOCUMENTATION_SITE_DIR` et `BPM_SCHEMA_CACHE_DIR` sont connues.

BPM peut être préparé pour le fonctionnement de la source à nœud unique dans l'état actuel. Cette évaluation ne constitue pas un certificat de renforcement de la production, une conception HA ou une acceptation de service packagé.

1. Enregistrez la source exacte et la configuration d'exécution.

Capturez `git rev-parse --short HEAD`, la version BPM, `BPM_HOST`, `BPM_PORT`, `BPM_DATABASE_URL`, `BPM_DOCUMENTATION_SITE_DIR`, `BPM_SCHEMA_CACHE_DIR` et indiquez si le processus est démarré avec `make dev` ou une commande `uvicorn app.main:app` appartenant directement.

2. Exécutez des contrôles d'activité et de préparation à partir de la même limite de réseau que les utilisateurs utiliseront.

```
export BPM_HOST="127.0.0.1"
export BPM_PORT="8000"
make dev
curl -fsS http://127.0.0.1:8000/health
curl -fsS http://127.0.0.1:8000/health/ready
```

Enregistrez le résultat comme preuve `GET /health` et `GET /health/ready`. Les sondes réussies prouvent uniquement la vivacité et la préparation du processus source.

3. Vérifiez les données et les preuves documentaires avant de partager l'instance.

Confirmez que `data/bpm.db` est la base de données prévue, qu'au moins un profil peut être exporté en tant que `polices.json` et que `/help/` ouvre l'artefact de documentation généré ou un état d'indisponibilité documenté.

4. Classifiez l'instance comme opération source à l'état actuel.

Avertissement :

N'étiquetez pas l'instance comme étant packagée, renforcée, à charge équilibrée ou hautement disponible. Si les contrôles externes ajoutent ces propriétés, enregistrez-les en tant que responsabilités externes et limitez les preuves BPM au fonctionnement de source à nœud unique.

La note de préparation indique ce qui peut être fait en toute sécurité aujourd'hui à partir de la source, quelles vérifications ont réussi et quelles décisions doivent attendre la distribution et le support d'exécution.

Utilisez la rubrique Exposition réseau et préparation au proxy avant d'exposer l'instance au-delà d'un poste de travail d'administrateur local ou d'un hôte de laboratoire contrôlé.

Enregistrer les limites HA et de production différées

Indiquez quelles fonctionnalités de production, HA, de proxy inverse, de restauration et de gestion des services sont absentes aujourd'hui et ce qui peut être préparé pour un travail de conception ultérieur.

Des notes sur la préparation, l'exposition du réseau, la sauvegarde/exportation, la surveillance et la fenêtre de mise à jour existent pour l'instance d'exécution source actuelle.

Cette rubrique enregistre la limite entre le fonctionnement actuel de la source à nœud unique et la future architecture de production. Il est délibérément explicite sur ce qui est prévu mais pas encore mis en œuvre.

1. Nommez la forme actuelle prise en charge.

Écrivez que la documentation Administrateur BPM/DevOps prend en charge le déploiement de sources, la configuration manuelle, les vérifications d'état/de préparation, les exemples d'API, les preuves de sauvegarde manuelle, les vérifications de reconstruction de la documentation et les fenêtres de mise à jour d'un seul nœud.

2. Nommer les capacités de service et de proxy absentes.

Répertoriez les services packagés, les unités de service Linux natives, les services Windows, les recettes officielles de proxy inverse, les recettes TLS/proxy, les secrets gérés, la configuration de la passerelle d'authentification et le renforcement de la production comme absents de la documentation source de BPM.

3. Nom absent HA et capacités de mise à niveau.

Liste du clustering HA pris en charge, de la coordination multi-nœuds, des conseils d'affinité de session d'équilibrage de charge, du verrouillage du stockage partagé, de la migration sans temps d'arrêt, des mises à niveau propagées, du basculement automatique et de l'automatisation officielle de la restauration en tant que travaux futurs, et non comportement actuel.

4. Convertissez les besoins futurs en intrants de conception.

Avertissement :

Ne placez pas plusieurs nœuds sources derrière un équilibreur de charge et appelez le résultat pris en charge HA. Capturez les RTO/RPO requis, la vérification des sauvegardes, la propriété de TLS, la propriété des secrets, la journalisation, la surveillance et les attentes en matière de fenêtre de mise à niveau en tant qu'entrées pour les tâches de distribution/d'exécution ultérieures.

L'enregistrement de limite empêche la production non prise en charge, les HA et les revendications de proxy inverse tout en préservant le travail de préparation utile pour les versions futures.

Utilisez cet enregistrement lors de la documentation manuelle QA et lorsque les futurs éléments d'installation, de service, de proxy inverse et HA sont approuvés.

Déployer BPM depuis les sources sous Linux

Préparer un hôte Linux pour le déploiement source

Confirmez les limites de déploiement source prises en charge, les prérequis de l'hôte et les capacités de production différée avant d'installer BPM à partir d'une extraction de référentiel.

Utilisez un hôte Linux sur lequel vous pouvez installer Python 3.14+, Git, créer des outils et curl. Ce runbook suppose une extraction de source, et non un programme d'installation packagé.

BPM peut être exécuté à partir des sources pour le développement, la révision et l'évaluation administrative contrôlée. Les programmes d'installation packagés, les unités de service natives, les recettes systemd officielles, les recettes de proxy inverse et TLS, le clustering HA, les secrets gérés et le renforcement de la production ne sont pas fournis dans cette version et ne doivent pas être présentés comme un comportement de déploiement pris en charge.

1. Identifiez l'objectif du déploiement et notez qu'il s'agit d'un déploiement Linux basé sur la source.

Résultat attendu : la note d'opérations indique que BPM est démarré à partir d'une extraction Git et que les installateurs spécifiques à la distribution sont différés.

2. Installez ou vérifiez les prérequis de l'hôte.

Les outils requis sont Git, Python 3.14+, pip, make, curl et un shell pouvant exécuter `source .venv/bin/activate`. Les commandes du référentiel utilisent l'environnement virtuel local et ne nécessitent pas de package BPM global.

3. Créez un répertoire de travail distinct des données et des secrets de production.

Conservez l'extraction du référentiel, `.venv` et les données locales SQLite sous un chemin contrôlé. Ne stockez pas de vrais secrets dans des exemples de documentation ou des fichiers validés.

4. Confirmez ce qui n'est pas intentionnellement couvert avant de continuer.

Avertissement :

Ce runbook ne fournit pas de programme d'installation packagé, de service natif, d'unité systemd prise en charge, de recette de proxy inverse, de terminaison TLS, de topologie HA, de magasin secret géré, d'automatisation de sauvegarde ou de liste de contrôle de renforcement de la production.

Les hypothèses d'hôte et de fonctionnement sont prêtes pour un processus d'extraction de source BPM pris en charge.

Continuez avec la rubrique sur l'extraction des sources Linux avant de configurer les variables d'exécution.

Configurer l'extraction des sources Linux

Clonez le référentiel BPM, créez l'environnement virtuel Python, installez les dépendances de développement et préparez le schéma de base de données locale.

L'hôte Linux est préparé et l'administrateur dispose de l'URL du dépôt BPM ainsi que de la branche, de la balise ou de la révision cible approuvée.

Le référentiel README définit la séquence de configuration source prise en charge. Utilisez les commandes locales du référentiel afin que le dépannage ultérieur puisse comparer l'environnement avec les scripts validés.

1. Clonez le référentiel et passez à la caisse.

```
git clone <repository-url> browser-policy-manager
cd browser-policy-manager
git checkout <approved-0.9.1-ref>
```

Remplacez les espaces réservés par l'URL du référentiel interne approuvé et la référence de la version.

2. Créez et activez l'environnement Python local.

```
python -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
```

L'environnement virtuel reste à l'intérieur de la caisse et n'est pas validé.

3. Installez les dépendances de la source BPM à partir de la caisse.

```
pip install -e ".[dev]"
```

L'installation modifiable maintient le code d'exécution et les tests alignés sur l'arborescence source actuelle.

4. Appliquez les migrations de base de données pour la base de données d'exécution sélectionnée.

```
alembic upgrade head
```

Avertissement :

Exécutez les migrations uniquement sur la base de données BPM prévue. Pour le processus source par défaut, il s'agit de la base de données locale SQLite et non d'une base de données de service de production.

La caisse dispose d'un environnement Python activé, de dépendances installées et d'un schéma de base de données migré.

Configurez l'environnement d'exécution avant de démarrer le processus d'application.

Configurer les paramètres d'exécution de la source Linux

Définissez l'URL de la base de données, l'hôte, le port, le mode de rechargement, CORS et le chemin du site de documentation utilisé par le processus BPM exécuté à la source.

Les dépendances d'extraction sont installées et les migrations ont été appliquées pour la base de données que vous prévoyez d'utiliser.

BPM lit les variables d'environnement avec le préfixe `BPM_` et prend également en charge un fichier `.env` local du référentiel. L'URL de base de données par défaut est `sqlite+aiosqlite:///./data/bpm.db`, qui stocke les données dans `data/bpm.db`.

1. Créez le répertoire de données local et définissez l'URL de la base de données.

```
mkdir -p data
export BPM_DATABASE_URL="sqlite+aiosqlite:///./data/bpm.db"
```

Pour les exécutions sources répétables, la même valeur peut être stockée dans un fichier `.env` local qui n'est pas validé.

2. Définissez les valeurs de réseau et de rechargement pour le processus source local.

```
export BPM_HOST="127.0.0.1"
export BPM_PORT="8000"
export BPM_RELOAD="true"
```

Utiliser le bouclage pour la vérification locale. La liaison à une interface partagée est une décision opérationnelle en dehors de cette limite source-runbook.

3. Vérifiez CORS et les paramètres du site de documentation.

Les valeurs par défaut de la source incluent `BPM_ENABLE_CORS`, `BPM_CORS_ALLOW_ORIGINS`, et `BPM_DOCUMENTATION_SITE_DIR`. Rester permissif CORS uniquement pour une évaluation locale contrôlée et point `BPM_DOCUMENTATION_SITE_DIR` sur un site de documentation statique installé lorsqu'il en existe un disponible.

4. Démarrez BPM à partir du script du référentiel.

```
make dev
```

Le Makefile exécute `uvicorn app.main:app --reload --port 8000`. Gardez le terminal de premier plan ouvert afin que les messages de démarrage et les traçages soient visibles.

BPM s'exécute à partir de la source avec des paramètres d'exécution locaux explicites.

Vérifiez l'état de santé, l'état de préparation, le produit UI et les premiers contrôles administratifs.

Vérifier un déploiement de source Linux

Utilisez des sondes d'intégrité, le Web UI, des journaux et une commande de test restreinte pour confirmer que l'instance BPM exécutée à la source est utilisable.

Le processus BPM s'exécute à partir de l'extraction du référentiel avec les paramètres BPM_DATABASE_URL, hôte, port et site de documentation prévus.

L'application actuelle expose l'activité à /health et l'état de préparation à /health/ready. Ces sondes confirment uniquement la réponse du processus en cours d'exécution ; ils ne prouvent pas le renforcement de la production, l'application de Firefox, la préparation à la sauvegarde ou le comportement de HA.

1. Vérifiez l'activité et l'état de préparation de l'hôte Linux.

```
curl -fsS http://127.0.0.1:8000/health
curl -fsS http://127.0.0.1:8000/health/ready
```

Les réponses attendues sont {"status": "ok"} et {"status": "ready", "ready": true}.

2. Ouvrez le produit UI et confirmez que la route de la bibliothèque répond.

Utilisez `http://127.0.0.1:8000/profiles`. La route doit charger l'interface BPM ou afficher un état d'application intentionnel, et non un traçage du serveur.

3. Inspectez les journaux d'application au premier plan.

Examinez le terminal exécutant `make dev` ou `uvicorn app.main:app --reload --port 8000`. Enregistrez les erreurs de démarrage, les erreurs de migration, les conflits de ports et les avertissements du site de documentation sans copier les secrets dans les tickets.

4. Consignez la limite du résultat de vérification.

Avertissement :

La réussite des sondes de santé, de la route UI et des journaux de premier plan ne certifie ni la préparation à la production, ni le comportement du proxy inverse, ni les sauvegardes gérées, ni l'application des politiques Firefox à l'exécution.

L'administrateur dispose de la preuve que le déploiement de la source Linux démarre, répond aux sondes d'intégrité/préparation, dessert la route UI et n'a aucun bloqueur immédiat de journaux de premier plan.

Si la vérification échoue, conservez ensemble la commande, les noms de variables d'environnement, les réponses de sonde et les lignes de journal pertinentes pour le dépannage. N'étendez pas ce runbook à des instructions d'installation différées ou de topologie de production.

Installer BPM depuis les sources sur Ubuntu 26.04 LTS

Installation complète de BPM 0.9.1 avec les paquets Python 3.14 de la distribution.

Utilisez Ubuntu 26.04 LTS avec sudo, HTTPS et une référence Git BPM 0.9.1 approuvée.

Cette procédure crée une exécution source à nœud unique contrôlée, pas un service système.

1. Vérifiez Ubuntu et installez les prérequis.

```
. /etc/os-release
test "$ID" = "ubuntu" && test "$VERSION_ID" = "26.04"
sudo apt-get update
sudo apt-get install -y build-essential ca-certificates curl git make python3.14 python3.14-dev
python3.14-venv
export BPM_PYTHON=/usr/bin/python3.14
"$BPM_PYTHON" --version
```

2. Récupérez la référence approuvée.

```
export BPM_REPOSITORY_URL="https://github.com/Goudron/browser-policy-manager.git"
export BPM_REF="<approved-0.9.1-ref>"
test "$BPM_REF" != "<approved-0.9.1-ref>"
cd "$HOME"
git clone "$BPM_REPOSITORY_URL" browser-policy-manager
cd "$HOME/browser-policy-manager"
git checkout --detach "$BPM_REF"
git status --short
git rev-parse --short HEAD
```

3. Créez l'environnement, installez BPM et migrez la base.

```
"$BPM_PYTHON" -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
pip install -e ".[dev]"
python --version
mkdir -p data
export BPM_DATABASE_URL="sqlite+aiosqlite:///./data/bpm.db"
alembic upgrade head
```

4. Démarrez BPM au premier plan.

```
export BPM_HOST="127.0.0.1"
export BPM_PORT="8000"
export BPM_RELOAD="true"
make dev
```

5. Vérifiez les sondes, l'interface et les journaux.

```
cd browser-policy-manager
source .venv/bin/activate
curl -fsS http://127.0.0.1:8000/health
curl -fsS http://127.0.0.1:8000/health/ready
curl -fsS -o /dev/null http://127.0.0.1:8000/profiles
```

6. Respectez la limite d'arrêt.

En cas d'échec, utilisez `Ctrl+C`, conservez les sorties et n'exécutez pas `alembic downgrade`.

La migration et la documentation sont terminées, les sondes et la Bibliothèque de profils répondent.

La séquence de commandes a réussi dans un conteneur d'espace utilisateur Ubuntu 26.04 propre. Cette preuve ne valide ni le démarrage natif, ni le noyau, ni `systemd`, ni le bureau, le matériel, le pare-feu de l'hôte ou le déploiement en production.

Installer BPM depuis les sources sur Debian 13.5

Installation complète avec CPython 3.14.6 utilisateur vérifié par somme de contrôle.

Utilisez Debian 13.5 avec sudo, HTTPS, 4 Go libres et une référence BPM approuvée.

Python 3.13 du système n'est pas remplacé ; Python 3.14.6 est installé sous \$HOME/.local.

1. Installez les prérequis de compilation.

```
. /etc/os-release
test "$ID" = "debian" && test "$VERSION_ID" = "13"
sudo apt-get update
sudo apt-get install -y build-essential ca-certificates curl git make libbz2-dev libffi-dev libgdbm-
dev liblzma-dev libncurses-dev libreadline-dev libsqlite3-dev libssl-dev tk-dev uuid-dev xz-utils
zlibg-dev
```

2. Compilez CPython 3.14.6 vérifié.

```
export PYTHON_VERSION="3.14.6"
export PYTHON_SHA256="143b1dddefaec3bd2e21e3b839b34a2b7fb9842272883c576420d605e9f30c63"
export PYTHON_PREFIX="$HOME/.local/python-$PYTHON_VERSION"
mkdir -p "$HOME/src" && cd "$HOME/src"
curl -fLO "https://www.python.org/ftp/python/$PYTHON_VERSION/Python-$PYTHON_VERSION.tar.xz"
printf '%s  %s\n' "$PYTHON_SHA256" "Python-$PYTHON_VERSION.tar.xz" | sha256sum -c -
tar -xf "Python-$PYTHON_VERSION.tar.xz"
cd "Python-$PYTHON_VERSION"
./configure --prefix="$PYTHON_PREFIX" --with-ensurepip=install
make -j"$(nproc)"
make altinstall
export BPM_PYTHON="$PYTHON_PREFIX/bin/python3.14"
"$BPM_PYTHON" --version
```

3. Récupérez BPM.

```
cd "$HOME"
export BPM_REPOSITORY_URL="https://github.com/Goudron/browser-policy-manager.git"
export BPM_REF="<approved-0.9.1-ref>"
test "$BPM_REF" != "<approved-0.9.1-ref>"
git clone "$BPM_REPOSITORY_URL" browser-policy-manager
cd browser-policy-manager
git checkout --detach "$BPM_REF"
git status --short
git rev-parse --short HEAD
```

4. Installez et migrez BPM.

```
"$BPM_PYTHON" -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
pip install -e ".[dev]"
```

```
python --version
mkdir -p data
export BPM_DATABASE_URL="sqlite+aiosqlite:///./data/bpm.db"
alembic upgrade head
```

5. Démarrez BPM.

```
export BPM_HOST="127.0.0.1"
export BPM_PORT="8000"
export BPM_RELOAD="true"
make dev
```

6. Vérifiez le résultat.

```
cd "$HOME/browser-policy-manager"
source .venv/bin/activate
curl -fsS http://127.0.0.1:8000/health
curl -fsS http://127.0.0.1:8000/health/ready
curl -fsS -o /dev/null http://127.0.0.1:8000/profiles
```

7. Arrêtez-vous sans risque.

Conservez la somme de contrôle et les journaux ; ne rétrogradez pas la base et ne supprimez pas Python avant le diagnostic.

Python 3.14.6, BPM, la migration, la documentation et les sondes sont prêts.

La séquence de commandes a réussi dans un conteneur d'espace utilisateur Debian 13.5 propre. CPython a signalé que les modules facultatifs `_dbm` et `_zstd` étaient indisponibles, mais le flux BPM complet a réussi. Cette preuve ne valide ni le démarrage natif, ni le noyau, ni systemd, ni le bureau, le matériel, le pare-feu de l'hôte ou le déploiement en production.

Installer BPM depuis les sources sur Fedora Linux 44

Installation complète de BPM 0.9.1 avec Python 3.14 de la distribution.

Utilisez Fedora 44 avec sudo, HTTPS et une référence BPM approuvée.

Il s'agit d'une exécution source à nœud unique, pas d'une recette systemd ou de production.

1. Installez les prérequis.

```
. /etc/os-release
test "$ID" = "fedora" && test "$VERSION_ID" = "44"
sudo dnf install -y ca-certificates curl gcc gcc-c++ git make python3 python3-devel
export BPM_PYTHON=/usr/bin/python3
"$BPM_PYTHON" --version
```

2. Récupérez BPM.

```
export BPM_REPOSITORY_URL="https://github.com/Goudron/browser-policy-manager.git"
export BPM_REF="<approved-0.9.1-ref>"
test "$BPM_REF" != "<approved-0.9.1-ref>"
cd "$HOME"
git clone "$BPM_REPOSITORY_URL" browser-policy-manager
cd "$HOME/browser-policy-manager"
git checkout --detach "$BPM_REF"
git status --short
git rev-parse --short HEAD
```

3. Installez et migrez BPM.

```
"$BPM_PYTHON" -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
pip install -e ".[dev]"
python --version
mkdir -p data
export BPM_DATABASE_URL="sqlite+aiosqlite:///./data/bpm.db"
alembic upgrade head
```

4. Démarrez BPM.

```
export BPM_HOST="127.0.0.1"
export BPM_PORT="8000"
export BPM_RELOAD="true"
make dev
```

5. Vérifiez le résultat.

```
cd browser-policy-manager
source .venv/bin/activate
curl -fsS http://127.0.0.1:8000/health
curl -fsS http://127.0.0.1:8000/health/ready
curl -fsS -o /dev/null http://127.0.0.1:8000/profiles
```

6. Arrêtez-vous sans risque.

Conservez les sorties DNF, pip, migration, construction et journaux ; ne rétrogradez pas la base.

BPM est migré, la documentation construite et les sondes répondent.

La séquence de commandes a réussi dans un conteneur d'espace utilisateur Fedora 44 propre. Cette preuve ne valide ni le démarrage natif, ni le noyau, ni systemd, ni le bureau, le matériel, le pare-feu de l'hôte ou le déploiement en production.

Installer BPM à partir des sources sur Linux Mint 22.3

Exécutez la séquence complète d'installation des sources de BPM 0.9.1 sur Linux Mint 22.3 en installant un CPython 3.14.6 vérifié par somme de contrôle sous l'utilisateur actuel.

Utilisez Linux Mint 22.3 (Zena) avec un accès sudo, HTTPS sortant, au moins 4 GB d'espace de construction libre et une réf. Git BPM 0.9.1 approuvée.

Linux Mint 22.3 utilise la base de packages Ubuntu Noble, dont Python par défaut est inférieur aux exigences 3.14 de BPM. Le système Python n'est pas remplacé.

1. Vérifiez Mint et installez les prérequis de build CPython.

```
. /etc/os-release
test "$ID" = "linuxmint" && test "$VERSION_ID" = "22.3"
sudo apt-get update
sudo apt-get install -y build-essential ca-certificates curl git make libbz2-dev libffi-dev libgdbm-
dev liblzma-dev libncurses-dev libreadline-dev libsqlite3-dev libssl-dev tk-dev uuid-dev xz-utils
zlibg-dev
```

2. Créez la version source CPython 3.14.6 vérifiée.

```
export PYTHON_VERSION="3.14.6"
export PYTHON_SHA256="143b1dddefaec3bd2e21e3b839b34a2b7fb9842272883c576420d605e9f30c63"
export PYTHON_PREFIX="$HOME/.local/python-$PYTHON_VERSION"
mkdir -p "$HOME/src" && cd "$HOME/src"
curl -fLO "https://www.python.org/ftp/python/$PYTHON_VERSION/Python-$PYTHON_VERSION.tar.xz"
printf '%s %s\n' "$PYTHON_SHA256" "Python-$PYTHON_VERSION.tar.xz" | sha256sum -c -
tar -xf "Python-$PYTHON_VERSION.tar.xz"
cd "Python-$PYTHON_VERSION"
./configure --prefix="$PYTHON_PREFIX" --with-ensurepip=install
make -j"$(nproc)"
make altinstall
export BPM_PYTHON="$PYTHON_PREFIX/bin/python3.14"
"$BPM_PYTHON" --version
```

3. Consultez la source BPM approuvée réf.

```
cd "$HOME"
export BPM_REPOSITORY_URL="https://github.com/Goudron/browser-policy-manager.git"
export BPM_REF="approved-0.9.1-ref"
test "$BPM_REF" != "<approved-0.9.1-ref>"
git clone "$BPM_REPOSITORY_URL" browser-policy-manager
cd browser-policy-manager
git checkout --detach "$BPM_REF"
git status --short
git rev-parse --short HEAD
```

4. Créez l'environnement, installez BPM et migrez.

```
"$BPM_PYTHON" -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
pip install -e ".[dev]"
python --version
mkdir -p data
export BPM_DATABASE_URL="sqlite+aiosqlite:///./data/bpm.db"
alembic upgrade head
```

5. Démarrez le BPM au premier plan.

```
export BPM_HOST="127.0.0.1"
export BPM_PORT="8000"
export BPM_RELOAD="true"
make dev
```

6. Vérifiez les sondes et le UI à partir d'un deuxième terminal.

```
cd "$HOME/browser-policy-manager"
source .venv/bin/activate
curl -fsS http://127.0.0.1:8000/health
curl -fsS http://127.0.0.1:8000/health/ready
curl -fsS -o /dev/null http://127.0.0.1:8000/profiles
```

7. Appliquer la limite d'arrêt et de restauration.

En cas d'échec, appuyez sur Ctrl+C, conservez les journaux et la sortie de la somme de contrôle, et n'exécutez pas `alembic downgrade`. Conservez le préfixe Python appartenant à l'utilisateur et effectuez la vérification jusqu'au diagnostic.

L'hôte Linux Mint 22.3 dispose d'un environnement d'exécution Python 3.14.6 appartenant à l'utilisateur, d'une extraction de source BPM migrée, d'une documentation construite, de sondes de passage, de UI de réponse et de journaux de premier plan.

La séquence de commandes a réussi dans un conteneur d'espace utilisateur Linux Mint 22.3 propre. CPython a signalé que les modules facultatifs `_dbm` et `_zstd` étaient indisponibles, mais le flux BPM complet a réussi. Cette preuve ne valide ni le démarrage natif, ni le noyau, ni systemd, ni le bureau, le matériel, le pare-feu de l'hôte ou le déploiement en production.

Installer BPM depuis les sources sur la branche stable de Manjaro

Installation complète après mise à jour intégrale de la branche stable et vérification de Python 3.14.

Utilisez la branche stable après la mise à jour du 2026-06-26 avec sudo, HTTPS et une référence BPM approuvée.

Manjaro est évolutive ; la branche et l'état entièrement mis à jour doivent être enregistrés.

1. Mettez à jour le système et installez les prérequis.

```
. /etc/os-release
test "$ID" = "manjaro"
test "$(pacman-mirrors -G)" = "stable"
sudo pacman -Syu --needed --noconfirm base-devel ca-certificates curl git python python-pip
pacman-mirrors -G
uname -r
export BPM_PYTHON=/usr/bin/python
"$BPM_PYTHON" --version
```

2. Récupérez BPM.

```
export BPM_REPOSITORY_URL="https://github.com/Goudron/browser-policy-manager.git"
export BPM_REF="<approved-0.9.1-ref>"
test "$BPM_REF" != "<approved-0.9.1-ref>"
cd "$HOME"
git clone "$BPM_REPOSITORY_URL" browser-policy-manager
cd "$HOME/browser-policy-manager"
git checkout --detach "$BPM_REF"
git status --short
git rev-parse --short HEAD
```

3. Installez et migrez BPM.

```
"$BPM_PYTHON" -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
pip install -e ".[dev]"
python --version
mkdir -p data
export BPM_DATABASE_URL="sqlite+aiosqlite:///./data/bpm.db"
alembic upgrade head
```

4. Démarrez BPM.

```
export BPM_HOST="127.0.0.1"
export BPM_PORT="8000"
```

```
export BPM_RELOAD="true"
make dev
```

5. Vérifiez le résultat.

```
cd browser-policy-manager
source .venv/bin/activate
curl -fsS http://127.0.0.1:8000/health
curl -fsS http://127.0.0.1:8000/health/ready
curl -fsS -o /dev/null http://127.0.0.1:8000/profiles
```

6. Arrêtez-vous sans risque.

Conservez les données de branche, paquets et journaux ; ne rétrogradez pas partiellement les paquets ou la base.

BPM et la documentation sont prêts et les sondes répondent.

La séquence de commandes a réussi dans un conteneur d'espace utilisateur de la branche stable Manjaro propre après enregistrement de la branche et de l'état complet des paquets. Cette preuve ne valide ni le démarrage natif, ni le noyau, ni systemd, ni le bureau, le matériel, le pare-feu de l'hôte ou le déploiement en production.

Déployer BPM depuis les sources avec Windows WSL

Préparer Windows 10/11 pour le déploiement de la source WSL

Activez WSL, choisissez une hypothèse de distribution Linux prise en charge et confirmez que BPM s'exécutera sous Linux plutôt qu'en tant que service Windows natif.

Utilisez Windows 10 ou Windows 11 avec autorisation pour activer WSL 2, installez une distribution Linux telle que Ubuntu LTS et exécutez des commandes à la fois dans PowerShell et dans le shell WSL.

Ce runbook documente le déploiement de la source BPM via WSL. Le processus de candidature pris en charge est toujours le flux de travail source Linux dans WSL ; Les programmes d'installation Windows natifs, les services Windows, les packages MSI, les tâches planifiées, l'hébergement IIS et la gestion des services Windows de production ne sont pas fournis dans cette version.

1. Confirmez la disponibilité WSL auprès de PowerShell.

```
wsl --status  
wsl --list --verbose
```

Résultat attendu : WSL est installé, WSL 2 est disponible et la distribution Linux choisie est visible.

2. Installez WSL et une distribution Linux approuvée si elles sont manquantes.

```
wsl --install  
wsl --install -d Ubuntu
```

Utilisez le nom de distribution approuvé par votre organisation. Redémarrez si Windows le demande, puis créez l'utilisateur Linux au premier démarrage de la distribution.

3. Enregistrez la distribution et les limites du système de fichiers.

Placez BPM sous le système de fichiers WSL Linux, par exemple /home/<linux-user>/browser-policy-manager. Évitez d'exécuter l'extraction à partir de /mnt/c, car la sémantique du système de fichiers Windows et l'analyse antivirus peuvent rendre les environnements Python, la surveillance des fichiers et les autorisations peu fiables.

4. Confirmez ce que ce chemin Windows ne prend pas encore en charge.

Avertissement :

Il ne s'agit pas d'une installation Windows native. Le runbook ne fournit pas de package MSI/EXE, de service Windows, de modèle de compte de service, d'intégration de l'Observateur d'événements, de stratégie de pare-feu Windows, de recette de proxy inverse/TLS, de topologie HA, de secrets gérés, d'automatisation de la sauvegarde ou de renforcement de la production.

Windows et WSL sont prêts pour un déploiement de source BPM basé sur Linux avec des limites claires en matière de plate-forme et de système de fichiers.

Continuez en ouvrant le shell de distribution WSL et en configurant l'extraction des sources sous le répertoire de base Linux. La documentation

Configurer la récupération de la source BPM dans WSL

Installez les prérequis Linux dans WSL, clonez BPM dans le système de fichiers Linux, créez l'environnement virtuel, installez les dépendances et appliquez les migrations.

Une distribution Linux WSL 2 est installée et ouverte, et l'administrateur dispose de l'URL du référentiel BPM approuvé et de la branche ou de la balise 0.9.1.

Toutes les commandes de cette rubrique s'exécutent dans le shell Linux WSL, sauf si l'étape indique explicitement PowerShell. Cela maintient la configuration alignée sur le runbook de déploiement source Linux et évite l'ambiguïté du chemin Windows natif.

1. Installez les prérequis Linux dans la distribution WSL.

```
sudo apt-get update
sudo apt-get install -y git make curl python3 python3-venv python3-pip
python3 --version
```

Utilisez le gestionnaire de packages et les noms de packages approuvés pour la distribution WSL sélectionnée.

2. Clonez BPM dans le système de fichiers Linux.

```
cd ~
git clone <repository-url> browser-policy-manager
cd browser-policy-manager
git checkout <approved-0.9.1-ref>
```

Ne placez pas la récupération de travail sous /mnt/c pour le processus source WSL pris en charge.

3. Créez et activez l'environnement Python local.

```
python -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
pip install -e ".[dev]"
```

Si la distribution expose Python uniquement en tant que python3, créez l'environnement avec `python3 -m venv .venv`, puis utilisez l'environnement activé.

4. Appliquez les migrations de bases de données dans la caisse WSL.

```
mkdir -p data
export BPM_DATABASE_URL="sqlite+aiosqlite:///./data/bpm.db"
alembic upgrade head
```

Avertissement :

Le fichier SQLite est un fichier Linux dans WSL. Ne modifiez pas et ne déplacez pas data/bpm.db depuis les outils Windows pendant que BPM est en cours d'exécution.

La récupération WSL comporte des prérequis Linux, un environnement virtuel local de référentiel, des dépendances installées et une base de données locale migrée.

Configurez les variables d'exécution et l'accès au navigateur avant de démarrer BPM.

Configurer le runtime WSL et l'accès au navigateur

Définissez les variables d'exécution BPM dans WSL, choisissez une adresse de liaison sécurisée, démarrez le processus source et identifiez l'URL du navigateur à partir de Windows.

La récupération BPM, l'environnement virtuel, les dépendances et les migrations sont prêts dans le système de fichiers WSL Linux.

BPM lit les variables d'environnement BPM_ du processus Linux. Le WSL moderne transmet normalement localhost à Windows, mais le WSL IP peut être vérifié lorsque l'accès au navigateur ou le comportement du pare-feu n'est pas clair.

1. Définissez les variables d'exécution locales dans le shell WSL.

```
source .env/bin/activate
export BPM_DATABASE_URL="sqlite+aiosqlite:///./data/bpm.db"
export BPM_HOST="0.0.0.0"
export BPM_PORT="8000"
export BPM_RELOAD="true"
```

BPM_HOST="0.0.0.0" permet au navigateur Windows d'atteindre le processus WSL via le transfert localhost. Utilisez 127.0.0.1 uniquement lorsque le navigateur est également dans WSL ou que le transfert a été vérifié.

2. Démarrez BPM à partir de la caisse WSL.

```
make dev
```

Le Makefile démarre uvicorn app.main:app --reload --port 8000. Gardez le terminal WSL ouvert pour les messages de démarrage, les traçages et la sortie de l'observateur de fichiers.

3. Identifiez l'URL du navigateur Windows.

```
hostname -I
```

Essayez d'abord `http://localhost:8000/profiles` depuis Windows. Si le transfert localhost n'est pas disponible, utilisez le premier WSL IP signalé par `hostname -I`, par exemple `http://<wsl-ip>:8000/profiles`.

4. Gardez les limites de sécurité explicites.

Avertissement :

L'ouverture du port WSL sur d'autres hôtes, la modification des règles de pare-feu Windows, l'ajout de TLS ou le placement d'un proxy inverse devant BPM ne relèvent pas de cette rubrique de déploiement de sources WSL et doivent attendre les conseils de préparation à la production.

Le processus source BPM s'exécute dans WSL et dispose d'un chemin d'accès au navigateur Windows identifié.

Vérifiez l'état, la préparation, l'accès UI et les modes de défaillance spécifiques à WSL.

Vérifier le déploiement de la source BPM via WSL

Vérifiez les sondes de WSL et de Windows, confirmez la route UI, inspectez les journaux et résolvez les problèmes courants de réseau et de système de fichiers WSL.

BPM s'exécute à partir de la caisse WSL avec les paramètres BPM_DATABASE_URL, hôte, port et site de documentation prévus.

La vérification doit prouver à la fois la réponse du processus Linux et le chemin d'accès du navigateur Windows. Il ne certifie pas l'installation native de Windows, le renforcement de la production, le comportement du proxy inverse, la préparation à la sauvegarde ou l'application de l'exécution de Firefox.

1. Vérifiez l'état de santé et l'état de préparation dans WSL.

```
curl -fsS http://127.0.0.1:8000/health
curl -fsS http://127.0.0.1:8000/health/ready
```

Les réponses attendues sont {"status": "ok"} et {"status": "ready", "ready": true}.

2. Vérifiez les mêmes sondes depuis Windows PowerShell.

```
Invoke-WebRequest http://localhost:8000/health
Invoke-WebRequest http://localhost:8000/health/ready
```

Si localhost ne fonctionne pas, réessayez avec le WSL IP de hostname -I. Enregistrez quelle adresse a réussi.

3. Ouvrez la route UI depuis le navigateur Windows.

Utilisez `http://localhost:8000/profiles` ou `http://<wsl-ip>:8000/profiles`. La route de la bibliothèque doit charger l'interface BPM ou un état vide/erreur intentionnel, et non un traçage.

4. Résolvez les échecs spécifiques à WSL avant de modifier le code BPM.

Pour les échecs de connexion, confirmez que `make dev` est toujours en cours d'exécution, vérifiez `BPM_HOST`, exécutez `wsl --list --verbose` et actualisez le WSL IP après avoir redémarré la distribution. En cas d'erreur de surveillance lente des fichiers ou d'autorisation, déplacez l'extraction de `/mnt/c` dans `/home/<linux-user>`. Courir uniquement une fois que le chemin d'accès à l'environnement et l'itinéraire réseau sont stables.

Avertissement :

La réussite des vérifications d'état via WSL ne prouve pas la préparation à la production, le comportement du service Windows, les sauvegardes gérées ou la prise en charge de HA.

L'administrateur a la preuve que BPM démarre dans WSL, répond aux sondes, est accessible depuis Windows et n'a aucun bloqueur immédiat de réseau WSL ou de système de fichiers.

Conservez ensemble la sortie PowerShell, les commandes du shell WSL, les noms de variables BPM_*, les réponses de sonde et les journaux de premier plan pour le dépannage sans copier les secrets.

Exploiter et mettre à jour BPM depuis les sources

Examiner les sources de configuration de DevOps

Identifiez les sources de configuration BPM actuelles, enregistrez les valeurs d'exécution efficaces et séparez les paramètres ordinaires des attentes de gestion des secrets non prises en charge.

BPM est déployé à partir de la source sous Linux ou via WSL, et l'administrateur peut inspecter l'environnement shell, le fichier `.env` facultatif et l'extraction de la source.

BPM charge les paramètres des variables d'environnement avec le préfixe `BPM_` et d'un fichier racine du référentiel `.env`. Le modèle de configuration réside dans les valeurs par défaut documentées. Cette version ne définit pas de magasin secret d'application, de coffre-fort d'informations d'identification, de paramètre de jeton API, de paramètre de clé de chiffrement ou de profil de configuration de production.

1. Répertoirez les sources de configuration utilisées pour cette exécution.

Enregistrez si les valeurs proviennent d'exportations shell, d'un `.env` local ou de valeurs par défaut dans les valeurs par défaut documentées. Gardez `.env` local à la caisse et ne le validez pas.

2. Enregistrez les paramètres d'exécution qui affectent le comportement du processus.

Inclut `BPM_APP_NAME`, `BPM_APP_VERSION`, `BPM_DEBUG`, `BPM_HOST`, `BPM_PORT`, `BPM_RELOAD`, `BPM_API_PREFIX`, `BPM_DEFAULT_LOCALE`, `BPM_SUPPORTED_LOCALES`, `BPM_I18N_DIR` et `BPM_DOCUMENTATION_SITE_DIR`. Les valeurs par défaut sont les valeurs par défaut de la source locale, et non la politique de production.

3. Enregistrez séparément les données, le schéma et les paramètres liés au réseau.

Suivez `BPM_DATABASE_URL`, `BPM_DB_ECHO`, `BPM_ENABLE_CORS`, `BPM_CORS_ALLOW_ORIGINS`, `BPM_SCHEMA_BASE_URL`, `BPM_SCHEMA_CACHE_DIR` et `BPM_SCHEMA_HTTP_TIMEOUT`. Traitez la base de données URL et les origines du réseau comme des preuves opérationnelles même lorsqu'elles ne contiennent aucun mot de passe.

4. Gardez les secrets des exemples de configuration BPM.

Avertissement :

La documentation de déploiement source de BPM ne fournit pas de configuration d'authentification, de secrets gérés, de configuration de chiffrement au repos, de rotation de jetons, d'isolation de locataire ou de garanties de rédaction de secrets. Si votre environnement a besoin de ces contrôles, enregistrez-les en tant que contrôles de plateforme externes plutôt qu'en tant que fonctionnalités prises en charge par BPM.

Le déploiement dispose d'un inventaire de configuration révisé qui correspond aux paramètres source actuels et n'implique pas un comportement de gestion des secrets non pris en charge.

Utilisez les rubriques de stockage, de réseau et de limites opérationnelles pour examiner l'impact des valeurs enregistrées avant d'exposer BPM au-delà d'un environnement local contrôlé.

Planifier le stockage, les journaux et les preuves de sauvegarde

Localisez la base de données de déploiement source, le cache de schéma, le site de documentation, les journaux de premier plan et les preuves de sauvegarde manuelle sans revendiquer la prise en charge de la sauvegarde ou de la restauration gérée.

L'inventaire de configuration de déploiement comprend `BPM_DATABASE_URL`, `BPM_SCHEMA_CACHE_DIR` et `BPM_DOCUMENTATION_SITE_DIR`.

Le déploiement source par défaut stocke les données d'application dans SQLite à `sqlite+aiosqlite:///./data/bpm.db`, correspondant à `data/bpm.db` sous la racine du référentiel. Les fichiers de cache de schéma se trouvent par défaut sous `app/schemas/mozilla` et un artefact de documentation installé est lu à partir de `app/documentation/site` lorsqu'il est présent.

1. Identifiez les fichiers et répertoires qui détiennent l'état opérationnel.

Enregistrez le chemin du référentiel, `data/`, `data/bpm.db`, `app/schemas/mozilla` et `app/documentation/site`. Dans WSL, conservez ces chemins dans le système de fichiers Linux plutôt que sous `/mnt/c`.

2. Définir une étape de sauvegarde manuelle des preuves avant les travaux risqués.

Arrêtez le processus BPM de premier plan, copiez `data/bpm.db` dans un emplacement de sauvegarde contrôlé et enregistrez l'horodatage, la révision de la source, l'URL de la base de données, la taille du fichier et la somme de contrôle. BPM ne fournit pas de sauvegarde automatisée, de récupération ponctuelle ou d'orchestration de restauration testée.

3. Capturez les journaux du processus source actuel.

La source exécute l'écriture des messages de démarrage, la sortie d'accès Uvicorn, les traces, les erreurs de migration, les conflits de port et les avertissements du site de documentation sur le terminal de premier plan, `stdout` ou `stderr`. Aucun répertoire de journaux d'application BPM dédié, stratégie de rotation des journaux, demande ID, journal d'audit ou intégration de l'Observateur d'événements n'est documenté dans cette version.

4. Associez les preuves de sauvegarde de fichiers aux preuves d'exportation lorsque le transfert de profil est important.

Utilisez des exemples d'exportation d'API existants pour exporter les documents Firefox `policies.json` pertinents et stocker l'état de la réponse, le profil ID, la révision, le canal de schéma et le hachage du document exporté. Ne traitez pas les preuves d'exportation comme une restauration complète de la base de données.

L'administrateur sait où se trouve l'état actuel du déploiement de la source et quelles preuves manuelles peuvent être collectées avant les mises à jour ou le dépannage.

Avant de modifier les révisions sources ou les schémas de base de données, utilisez le futur runbook de mise à jour à partir de la source et arrêtez-vous lorsque les preuves de sauvegarde ou de restauration sont incomplètes.

Examiner le réseau, CORS et les limites de sécurité

Vérifiez l'hôte, le port, le rechargement, les sondes d'intégrité, les valeurs par défaut de CORS et les limites de sécurité avant de rendre accessible une instance BPM exécutée à la source.

Le processus source peut être démarré avec `make dev` et l'administrateur peut modifier `BPM_HOST`, `BPM_PORT`, `BPM_RELOAD`, `BPM_ENABLE_CORS` et `BPM_CORS_ALLOW_ORIGINS`.

Les valeurs par défaut actuelles sont optimisées pour le fonctionnement de la source locale : `BPM_HOST="0.0.0.0"`, `BPM_PORT="8000"`, `BPM_RELOAD="true"`, `BPM_ENABLE_CORS="true"` et les origines permissives équivalentes à `["*"]`. Ces valeurs par défaut ne constituent pas une politique renforcée d'exposition du réseau.

1. Choisissez l'adresse de liaison la plus étroite pour l'environnement actuel.

Utilisez `127.0.0.1` pour les vérifications Linux locales. Utilisez `0.0.0.0` uniquement lorsque le transfert du navigateur WSL ou un réseau de laboratoire contrôlé l'exige. Enregistrez la raison de toute liaison sans bouclage.

2. Bénéficiez d'un accès opérationnel avec des sondes d'intégrité et de préparation.

Utilisez `GET /health` et `GET /health/ready` avant les exemples d'API ou les contrôles de fumée UI. Les sondes réussies montrent uniquement la vivacité/l'état de préparation du processus ; ils ne prouvent pas l'état de sauvegarde de la base de données, l'application de Firefox, l'authentification, l'autorisation ou la topologie de production.

3. Restreindre CORS pour toute évaluation partagée.

Pour les expériences locales, le permissif `BPM_CORS_ALLOW_ORIGINS` peut être acceptable. Pour un hôte partagé, définissez des origines explicites et enregistrez à qui appartient l'origine du navigateur. La documentation BPM ne définit pas les protections authentifiées CORS, CSRF, l'authentification API ou l'isolation des locataires.

4. Gardez les contrôles de sécurité non pris en charge hors de la revendication BPM.

Avertissement :

Cette version ne fournit pas de terminaison TLS, de recettes de proxy inverse, de politique de pare-feu, de limitation de débit, de signature de requêtes, de journalisation d'audit, de configuration SSO, OAuth/OIDC, de contrôle d'accès basé sur les rôles, d'analyse des vulnérabilités ou de basculement HA. Documentez-les en tant que contrôles externes si votre environnement les ajoute.

La posture du réseau BPM exécuté à la source est documentée comme une exposition contrôlée locale ou en laboratoire plutôt que comme une architecture de sécurité de production.

Si les sondes échouent ou si l'exposition n'est pas claire, utilisez les rubriques de vérification du déploiement source avant de continuer avec les exemples d'intégration d'API.

Enregistrer les limites source-opération

Créez une note de transfert d'opérations indiquant l'utilisateur d'exécution, le modèle de processus, les contrôles pris en charge et les capacités de production différées.

Les examens de la configuration, du stockage, de la sauvegarde, du réseau et de CORS ont été effectués pour l'instance exécutée à la source.

BPM Administrator/DevOps prend actuellement en charge les opérations d'exécution à la source pour une évaluation contrôlée. Il ne fournit pas encore d'installateurs de distribution, de services natifs, de démonisation officielle, de topologie de production ou de guides d'exploitation opérationnels complets pour chaque contrôle d'entreprise.

1. Enregistrez l'utilisateur d'exécution et le modèle de processus.

Identifiez l'utilisateur shell propriétaire de la caisse, `.venv`, `data/` et le processus `make dev` ou `uvicorn app.main:app --reload --port 8000` en cours d'exécution. Ne revendiquez pas un compte de service, une unité `systemd`, un service Windows ou une politique de redémarrage pris en charge, à moins que votre plate-forme externe ne les fournisse et ne les possède.

2. Enregistrez les contrôles pris en charge aujourd'hui.

Inclut la révision de la source, les valeurs de configuration actives, l'état de la migration, la fumée de route `/health`, `/health/ready`, UI, l'état de l'exemple d'API, l'examen du journal de premier plan et les preuves de sauvegarde manuelle. Liez les preuves d'API à des exemples d'API réutilisables plutôt que de dupliquer les procédures de point de terminaison.

3. Enregistrez les capacités différées comme bloqueurs ou responsabilités externes.

Appelez les programmes d'installation packagés, les services natifs Linux ou Windows, les recettes de proxy inverse et TLS, le clustering HA, les secrets gérés, la configuration de l'authentification, l'automatisation des sauvegardes, l'orchestration des restaurations, les tableaux de bord de surveillance, la rotation des journaux et le renforcement de la production, non fournis par la documentation source de BPM.

4. Joignez des références de récupération destinées aux utilisateurs sans déplacer la propriété.

Lorsqu'un opérateur a besoin de comprendre les symptômes de UI, créez un lien vers un dépannage destiné à l'utilisateur, tel que la récupération de la connexion du produit. Conservez le déploiement, l'API, la sauvegarde et la propriété du réseau dans le Guide de l'administrateur/DevOps.

La note de transfert indique ce que le déploiement source actuel peut prendre en charge, quelles preuves le prouvent et quelles responsabilités de production restent en dehors de BPM.

Utilisez cette note de limite comme point de départ pour le runbook de mise à jour à partir de la source et les tâches ultérieures de documentation de production/HA/proxy inverse.

Préparer les éléments de preuve avant une mise à jour depuis les sources

Collectez la révision source, la configuration, la sauvegarde de la base de données, les preuves d'exportation de profil et les conditions d'arrêt avant de déplacer BPM vers une révision source plus récente.

BPM s'exécute à partir d'une extraction de source Linux ou WSL et l'administrateur a terminé les examens de configuration, de stockage, de sauvegarde et des limites opérationnelles.

Une procédure de mise à jour à partir de la source modifie le code, les dépendances, le schéma de base de données et les attentes de sortie de la documentation. BPM ne fournit pas de restauration automatique, de garantie de rétrogradation de base de données, de sauvegarde gérée ou d'outils de mise à niveau packagés. Considérez la collecte de preuves comme une étape préalable à la mise à jour.

1. Arrêtez le processus BPM de premier plan et enregistrez l'état actuel de la source.

```
git status --short
git rev-parse --short HEAD
```

Continuez uniquement lorsque l'état de l'arborescence de travail est compris et que la révision source actuelle est enregistrée comme référence de restauration.

2. Enregistrez les chemins de configuration et d'exécution efficaces.

Capturez les `BPM_DATABASE_URL`, `BPM_DOCUMENTATION_SITE_DIR`, `BPM_SCHEMA_CACHE_DIR`, `BPM_HOST`, `BPM_PORT` actifs et indiquez si les valeurs proviennent d'exportations shell ou de `.env`. Ne copiez pas les secrets dans la note de mise à jour.

3. Créez une preuve de sauvegarde manuelle de la base de données lorsque BPM est arrêté.

```
mkdir -p backups
cp data/bpm.db backups/bpm-pre-update.db
sha256sum backups/bpm-pre-update.db
```

Enregistrez l'horodatage, la révision actuelle, l'URL de la base de données, la taille du fichier et la somme de contrôle. Si le déploiement n'utilise pas `data/bpm.db`, sauvegardez plutôt le chemin résolu à partir de `BPM_DATABASE_URL`.

4. Exportez les preuves de profil critiques avant de modifier le code.

Utilisez les exemples d'exportation d'API existants pour enregistrer les documents Firefox `policies.json` requis et enregistrer l'état de la réponse, le profil ID, la révision, le canal de schéma et le hachage du document exporté.

Avertissement :

Si la sauvegarde de la base de données ou la preuve d'exportation est manquante, arrêtez la mise à jour. Ne remplacez pas les preuves de sauvegarde manquantes par l'hypothèse que la restauration de Git ou alembic downgrade sera sûre.

La mise à jour dispose d'un ensemble de preuves vérifiées avant modification et d'une référence source précédente connue.

Procédez à l'actualisation de la révision source et des dépendances uniquement une fois l'ensemble de preuves terminé et examiné.

Actualiser la révision source et les dépendances

Déplacez la récupération vers une révision plus récente approuvée et réinstallez l'environnement Python sans masquer les échecs de dépendance ou d'arborescence de travail.

L'ensemble de preuves préalable à la mise à jour existe, la révision précédente est enregistrée et une branche, une balise ou une validation cible approuvée est connue. Les mises à jour des sources

Les mises à jour de la source BPM sont des opérations dans les environnements Git et Python effectuées dans la caisse existante. La procédure n'installe pas de distribution packagée, de service natif ou de gestionnaire de mise à niveau de production.

1. Récupérez la référence source approuvée.

```
git fetch --tags
git status --short
```

Arrêtez-vous si les modifications locales sont inattendues. Conservez les modifications appartenant à l'utilisateur ou au site au lieu de les écraser.

2. Passer à la révision source la plus récente approuvée.

```
git checkout <approved-new-ref>
git rev-parse --short HEAD
```

Enregistrez la révision cible et comparez-la avec le ticket de modification approuvé ou la note de version.

3. Actualisez l'environnement Python local du référentiel.

```
source .venv/bin/activate
python -m pip install --upgrade pip
pip install -e ".[dev]"
```

Utilisez le `.venv` existant à moins que la note de mise à jour n'exige explicitement sa reconstruction. Enregistrez les échecs de dépendance avant de modifier la base de données.

4. Arrêt en cas d'échecs de source ou de dépendance non résolus.

Avertissement :

Si l'extraction, l'installation des dépendances ou l'activation de l'environnement virtuel échoue, n'exécutez pas de migrations. Revenez à la révision précédente enregistrée uniquement si la base de données n'a pas été modifiée, puis réinstallez l'ensemble de dépendances précédent et réexécutez les contrôles de fumée avant la mise à jour.

L'environnement de paiement et Python se trouve sur la révision cible approuvée, ou la mise à jour s'est arrêtée avant les modifications de la base de données.

Exécutez les migrations de bases de données et les vérifications de documentation uniquement une fois l'actualisation des dépendances réussie.

Exécuter les migrations et les vérifications de mise à jour de la source

Appliquez les migrations de base de données et vérifiez le déploiement BPM depuis la source après un changement de révision.

L'extraction et les dépendances ont été actualisées avec succès et les preuves de sauvegarde de la base de données ont été collectées alors que BPM était arrêté.

Les mises à jour peuvent introduire des migrations de base de données. BPM documente la migration vers l'avant avec `alembic upgrade head` ; aucune rétrogradation automatique ni restauration sûre du schéma n'est garantie.

1. Appliquez les migrations de base de données à la base de données souhaitée.

```
export BPM_DATABASE_URL="sqlite+aiosqlite:///./data/bpm.db"
alembic upgrade head
```

Utilisez la valeur de déploiement réelle pour `BPM_DATABASE_URL`. Arrêtez-vous immédiatement si le résultat de la migration n'est pas clair ou échoue.

2. Démarrez le processus BPM mis à jour et vérifiez son interface actuelle.

Vérifiez l'activité et l'état de préparation, puis ouvrez la route Bibliothèque. Si la documentation est requise pour le déploiement, confirmez que `/help/` signale un artefact installé prêt plutôt qu'un état absent ou incompatible.

3. Arrêtez-vous en cas d'échec de migration ou de vérification à l'exécution.

Avertissement :

Après une migration, ne présumez pas qu'une restauration de l'extraction Git suffit. N'exécutez `alembic downgrade` que si une note de version examinée confirme explicitement la prise en charge de la paire de révisions exacte. Lorsqu'une restauration est nécessaire, utilisez la sauvegarde de la base de données antérieure à la mise à jour comme base de récupération sûre.

La révision source mise à jour a terminé les migrations et les vérifications à l'exécution, ou la mise à jour s'est arrêtée à un point de défaillance nommé.

Démarrez BPM et exécutez la rubrique de vérification post-mise à jour avant de rendre l'instance aux utilisateurs ou aux intégrateurs.

Vérifier la mise à jour et appliquer les conditions d'arrêt de restauration

Démarrez le processus BPM mis à jour, vérifiez les sondes, UI, les preuves API et l'état de la documentation, puis décidez d'accepter, d'arrêter ou de restaurer à partir d'une sauvegarde.

Les vérifications des migrations, de la source de fumée et de la reconstruction de la documentation sont terminées ou un point de défaillance a été enregistré.

La vérification post-mise à jour prouve que l'exécution source mise à jour est utilisable pour l'environnement contrôlé actuel. Cela ne prouve pas la préparation à la production, le comportement du proxy inverse, HA, la sauvegarde gérée ou l'application de l'exécution de Firefox.

1. Démarrez le processus source mis à jour et vérifiez les sondes.

```
make dev
curl -fsS http://127.0.0.1:8000/health
curl -fsS http://127.0.0.1:8000/health/ready
```

Les réponses attendues restent {"status": "ok"} et {"status": "ready", "ready": true}.

2. Vérifiez le produit UI, les preuves API et l'état de la documentation.

Ouvrez <http://127.0.0.1:8000/profiles>, exécutez les exemples d'API réutilisables requis, confirmez les preuves `polices.json` exportées si nécessaire et vérifiez que `/help/` ne signale pas de documentation obsolète, incompatible ou manquante pour l'artefact installé.

3. Acceptez la mise à jour uniquement lorsque chaque vérification requise est enregistrée.

Enregistrez la révision précédente, la révision cible, le résultat de la migration, le résultat de la fumée source, le résultat de la validation/construction de la documentation, les réponses d'intégrité/préparation, le résultat de la fumée UI, les preuves d'API/d'exportation et tout contrôle de production différé connu.

4. Appliquez des conditions de restauration ou d'arrêt sans inventer la récupération automatique.

Si un échec se produit avant les migrations, revenez à la révision source précédente et réinstallez les dépendances. Si un échec se produit après les migrations, arrêtez BPM, restaurez la sauvegarde de base de données vérifiée avant la mise à jour, revenez à la révision source précédente, réinstallez les dépendances et répétez les vérifications d'état/UI. Si des preuves de sauvegarde sont manquantes, si la restauration n'est pas vérifiée ou si les données ont été modifiées après la migration, arrêtez et faites remonter au lieu de prétendre que la restauration a réussi.

Avertissement :

Ne marquez pas la mise à jour comme terminée lorsque les vérifications échouent, que la documentation est obsolète, que les exportations diffèrent de manière inattendue, que les migrations sont incertaines ou que les preuves d'annulation sont incomplètes.

La mise à jour est soit acceptée avec des preuves complètes, soit arrêtée auprès d'un bloqueur connu, soit restaurée à partir de la sauvegarde préalable à la mise à jour avec une preuve explicite.

Utilisez les preuves obtenues dans la note de transfert des opérations et dans la planification future de la production/HA/proxy inverse.

Vérifier l'état et la préparation du BPM

Utilisez les contrôles de fonctionnement et de disponibilité actuels comme une vérification préalable de l'état opérationnel légère avant les processus API.

Définissez `$BPM_BASE_URL`. Pour la configuration de l'hôte, le démarrage du service, les journaux, les mises à jour à partir de la source et les détails du système d'exploitation, utilisez le Guide de l'administrateur de déploiement source ; cette rubrique API documente uniquement la négociation d'intégration.

Cette tâche couvre API -HEALTH-001 et API -HEALTH-002. BPM expose des sondes sans préfixe sur `/health` et `/health/ready`. BPM n'expose pas les diagnostics de dépendance via ces sondes ; ils ne sont pas versionnés et ne remplacent pas la politique de nouvelle tentative ou d'incident spécifique au produit.

1. Appelez le contrôle de fonctionnement avant de démarrer une courte séquence API.

Exemple de requête : `GET $BPM_BASE_URL/health`. Exemple de réponse : `200 {"status": "ok"}`.

Exemple d'erreur : aucune erreur d'application n'est actuellement déclarée ; les échecs de transport, de passerelle ou de processus sont traités en dehors de BPM.

2. Appelez la contrôle de disponibilité avant la fin des opérations qui dépendent du démarrage de l'application.

Exemple de requête : `GET $BPM_BASE_URL/health/ready`. Exemple de réponse : `200`

`{"status": "ready", "ready": true}`. Exemple d'erreur : aucun corps d'état dégradé ou de détail de dépendance n'est actuellement exposé.

3. Utilisez la préparation comme une porte, et non comme un inventaire complet des dépendances.

Lorsque `ready` est `true`, le produit d'intégration peut poursuivre les appels de découverte, de validation, d'importation, d'exportation ou de cycle de vie de profil. Il ne doit pas déduire de cette réponse les détails sur l'état de la base de données, les détails du cache de schéma, l'état du portail de documentation, l'état de la file d'attente ou la topologie de déploiement.

4. Continuez à interroger et réessayez les décisions dans le produit d'intégration.

BPM ne déclare pas d'intervalle d'interrogation, d'en-tête de nouvelle tentative, d'objectif de niveau de service, de contrat de disjoncteur, de demande ID ou de webhook pour les modifications de sonde. Stockez votre propre horodatage, votre statut observé, votre délai d'attente et votre décision de réessayer.

5. Séparez les vérifications de prise de contact de l'API du dépannage du déploiement source.

Si les sondes sont inaccessibles ou si la réponse diffère des exemples, passez au Guide de l'administrateur de déploiement source pour les vérifications des processus, de l'environnement, de la base de données, des journaux et de la mise à jour à partir de la source au lieu d'étendre cette tâche d'intégration d'API dans les instructions de déploiement.

6. Enregistrez le résultat de la négociation opérationnelle avec l'action API en aval.

Avertissement :

Une réponse d'intégrité ou de préparation réussie n'authentifie pas les appelants, n'autorise pas les opérations, ne réserve pas les révisions de profil, ne garantit pas le succès ultérieur de l'API ou ne prouve pas l'application de l'exécution de Firefox. Traitez-le comme un petit signal de vol en amont.

Le produit d'intégration dispose d'une vérification préalable de l'état opérationnel BPM minimale et actuelle pour assurer sa vivacité et sa préparation sans dépendre d'un comportement de déploiement non documenté.

Après une vérification de préparation réussie, poursuivez la découverte OpenAPI ou la rubrique spécifique du processus API pour les opérations de validation, d'importation/exportation ou de cycle de vie des profils.

Intégrer BPM par l'API

Audience d'intégration et limite de l'API

La section d'intégration de l'API du Guide d'administration/DevOps est destinée aux outils externes qui utilisent l'API actuelle de BPM sans supposer de garanties non prises en charge.

Public

Utilisez ce guide lorsque vous créez ou examinez un produit de configuration, de conformité, d'orchestration, d'inventaire ou de contrôle qui échange des données de profil et de stratégie Firefox avec BPM. Les principaux lecteurs sont les intégrateurs d'API, les réviseurs de sécurité, les ingénieurs de versions et les responsables qui ont besoin d'un contrat reproductible pour la synchronisation des profils, les portes de validation, l'importation/exportation de politiques, les métadonnées de conformité, l'inventaire et les contrôles de santé.

Limite des preuves

La section repose sur le contrat OpenAPI généré, l'inventaire d'API maintenu, le code des routes et des modèles ainsi que des tests d'API ciblés. Elle documente l'API actuellement exposée par BPM. Elle n'autorise ni nouveaux points de terminaison, ni modèles de sécurité, ni schémas de données, ni sémantique de nouvelle tentative, ni promesse de compatibilité.

Rôles de consommateur pris en charge

Les consommateurs pris en charge sont des rôles génériques : gestionnaire de configuration, analyseur de conformité, porte de révision des stratégies, collecteur d'inventaire de profils, orchestrateur de déploiement et moniteur opérationnel. Ils peuvent appeler les opérations de service, de profil, d'import/export Firefox, de validation et de santé répertoriées dans l'inventaire de l'API.

Revendications non prises en charge

Ne décrivez pas un connecteur spécifique à un produit, une intégration de marché, une intégration certifiée, un partenariat, un agent géré ou un flux de travail de fournisseur à moins que ce connecteur ne soit réellement livré. Les routes HTML, les ressources statiques, le schéma généré interfaces et les assistants Python avec préfixe de soulignement ne sont pas une intégration machine stable API.

Propriété des données

BPM possède le modèle de profil normalisé, l'état de validation, la révision, l'état du cycle de vie, l'objet de conformité opaque et le rendu canonique d'exportation Firefox. Les produits d'intégration sont

propriétaires de leur planification, autorisation, piste d'audit, politique de nouvelle tentative, enregistrement d'exception, décision de déploiement et toute vérification d'exécution en dehors de BPM.

Modèles d'intégration pris en charge

BPM prend en charge les modèles d'API génériques pour les processus de profil, de validation, de document Firefox, de métadonnées, d'inventaire et d'intégrité.

Synchronisation du profil

Un produit d'intégration peut répertorier les profils, lire un profil par ID, créer un profil normalisé, corriger les champs de profil mutables, archiver, restaurer, supprimer définitivement ou réinitialiser la bibliothèque. La synchronisation doit traiter le profil ID et la révision comme un état appartenant à BPM et ne doit pas supposer d'opérations groupées, de mutation de nom ou de restauration automatique sur plusieurs requêtes.

Portes de validation

Une porte de validation peut soumettre un document canonique candidat `policies.json` au point de terminaison de validation pris en charge avant qu'un processus de déploiement ne l'accepte. La porte doit inspecter le corps de la réponse ainsi que le code d'état, car la validation peut renvoyer un statut HTTP réussi avec `ok=false`.

Importation et exportation de règles

Une intégration peut importer `Firefox policies.json` en tant que profil BPM et exporter un profil BPM enregistré vers `Firefox JSON canonique`. N'interchangez pas le modèle de profil normalisé avec la forme canonique du document Firefox ; l'import/export est la frontière entre ces représentations.

Transfert des métadonnées de conformité

L'API expose un objet `compliance` opaque lors des opérations de création, de mise à jour et d'importation de profil. Les intégrations peuvent transmettre leurs propres métadonnées via BPM, mais BPM ne publie pas de schéma versionné pour cet objet et ne valide pas la sémantique de conformité externe.

Inventaire et contrôles d'état

Les consommateurs de style inventaire peuvent répertorier les profils, demander des statistiques pour le même contexte de filtre, lire des profils individuels et appeler les points de terminaison de service, d'activité et de préparation. Les réponses d'intégrité sont intentionnellement compactes et n'exposent pas les diagnostics de dépendance, l'état de la documentation ou l'état de préparation au déploiement au-delà des champs fixes actuels.

Modèles non pris en charge

Le guide ne documente pas l'authentification, l'autorisation, les limites de débit, les clés d'idempotence, la demande identifiants, les webhooks, l'exportation en bloc, le CRUD en bloc, la pagination du curseur, les connecteurs spécifiques au produit, les partenariats avec les fournisseurs ou l'orchestration transactionnelle, car BPM n'expose pas ces garanties.

Conventions API

Utilisez ces conventions lors de la lecture des contrats OpenAPI BPM et de la création de demandes d'intégration.

Base URL et découverte

Exemples d'utilisation `$BPM_BASE_URL` comme espace réservé de déploiement. Découvrez le contrat en cours de `/openapi.json` et, pour une inspection interactive uniquement, le Schéma FastAPI interfaces à `/docs` et `/redoc`. Le futur portail de documentation produit est réservé aux `/help/`, donc ne traitez pas `/docs` comme Itinéraire de documentation DITA.

Types de contenu

La plupart des opérations de programmation utilisent des corps de requête et de réponse JSON. L'importation Firefox accepte `application/json` et `multipart/form-data`; les types de médias non pris en charge peuvent revenir 415. L'exportation de Firefox revient `application/json` et peut ajouter `Content-Disposition` lorsque la sortie de téléchargement est demandée.

Identifiants et cycle de vie

Profil identifiants sont des identifiants de chemin entiers appartenant à BPM. Conservez le retour id et revision après les opérations de création, d'importation, de lecture et de mise à jour. Les noms de profil sont uniques et ne peuvent pas être modifiés via `ProfileUpdate`. Les valeurs du cycle de vie sont actives, archivées et tout ; `include_deleted` est un indicateur de compatibilité qui doit être interprété avec le cycle de vie.

Modèles et canaux de schéma

`ProfileCreate`, `ProfileUpdate` et `ProfileRead` sont des formes de profil normalisées BPM. L'import/export Firefox utilise le `policies.json` canonique avec le `policies` de niveau supérieur. La validation utilise `ValidationRequest` et les canaux de schéma pris en charge tels que Firefox ESR et Release ; un mappage de politique simple reste un comportement de compatibilité, et non la forme externe préférée.

Filtrage, tri et pagination

Les opérations de liste de profils et de statistiques prennent en charge les filtres de recherche, de schéma, d'état de validation, de cycle de vie et d'inclusion-suppression. La pagination de la liste est offset/limite avec `limit=50` par défaut, 200 maximum et `offset=0`. Le tri utilise les champs décrits actuellement tels que `created_at`, `updated_at`, `name`, `schema_version` et `id`, avec l'ordre asc ou desc.

Statut et gestion des erreurs

Utilisez les codes d'état et la forme du corps ensemble. 200, 201 et 204 sont des familles de réussite, mais la validation peut renvoyer 200 avec `ok=false`. Les corps d'erreur utilisent FastAPI `detail` et peuvent contenir une chaîne, une liste de validation ou un objet BPM avec `message`, `error` ou `issues`.

Limites actuelles de l'API et contraintes de sécurité

BPM documente clairement les limitations actuelles afin que les intégrations ne déduisent pas de garanties qui ne sont pas exposées.

Contraintes de sécurité

L'API actuelle n'a pas de couche d'authentification ou d'autorisation. CORS est par défaut les origines *, sauf si les paramètres de déploiement le remplacent. La documentation ne doit pas impliquer l'authentification par jeton API, OAuth, le contrôle d'accès basé sur les rôles, l'isolation des locataires ou le comportement d'autorisation de production.

Versionnement et compatibilité

L'API n'a pas de préfixe de version, de stratégie de compatibilité, d'en-tête de dépréciation ou de version de type de support. Les intégrations doivent épingler la version BPM sur laquelle elles testent et surveiller `/openapi.json`, l'inventaire de l'API et les notes de version plutôt que de supposer une compatibilité ascendante garantie.

Concurrence, idempotence et nouvelle tentative

Seul PATCH prend en charge la concurrence optimiste via `expected_revision` en option. La création, l'importation, la suppression, la restauration, la réinitialisation et l'exportation n'exposent pas ETags, les en-têtes de précondition, les clés d'idempotence ou les nouvelles tentatives de contrats. Les consommateurs doivent concevoir leur propre comportement sécurisé de nouvelle tentative et de récupération de nom en double.

Limites des transactions et des volumes

BPM n'expose aucun point de terminaison de profil en bloc CRUD, aucun point de terminaison d'exportation en bloc, aucune pagination du curseur et aucun contrat de transaction multi-opérations. Une intégration en plusieurs étapes ayant échoué n'est pas automatiquement annulée par BPM ; le produit d'intégration possède le séquençage de récupération et les enregistrements d'audit.

Limites de l'enveloppe d'erreur

Les réponses d'erreur n'utilisent pas une enveloppe versionnée stable. Les statuts 400, 404, 409, 415, 422 et 503 peuvent porter différentes formes `detail`. Les consommateurs doivent se classer par opération, statut et forme corporelle documentée.

Surfaces exclues et HTML

Les six opérations OpenAPI web sont des routes de produits HTML, et non une intégration machine stable API. `/i18n/{locale}.json`, `/favicon.ico`, `/static/*`, le schéma généré interfaces et les assistants Python avec préfixe de trait de soulignement sont exclus des rubriques sur les opérations d'intégration.

Limites de réponse sanitaire

`/health` et `/health/ready` exposent uniquement les champs d'état compacts. L'état de préparation ne signale pas les diagnostics de dépendance, les détails de la base de données, l'état du schéma, l'état du portail de documentation ou l'approbation du déploiement. Les moniteurs opérationnels doivent les traiter comme des points de terminaison minimaux de prise de contact.

Réutiliser les exemples d'API et les modèles d'environnement DevOps

Utilisez des modèles de requêtes indépendants du langage, des modèles d'environnement, des commandes curl copiables, des exemples en plusieurs parties et des assertions Python compactes pour les flux de travail actuels de l'API BPM.

Définissez `BPM_BASE_URL` pour l'instance BPM testée. Ne placez pas les informations d'identification, les noms de locataires, les noms d'hôtes personnels, les hôtes privés ou le profil de production identifiants dans des exemples réutilisables ; BPM ne documente actuellement pas l'authentification API.

Cette tâche couvre API-HEALTH-001, API-HEALTH-002, API-PROFILE-001, API-PROFILE-003, API-PROFILE-004, API-PROFILE-005, API-FF-001, API-VAL-001 et API-FF-002. Les exemples utilisent intentionnellement `$BPM_BASE_URL`, des noms synthétiques, `release-153`, des assertions de réponse et un appareil jetable identifiants au lieu d'hôtes ou de secrets spécifiques à l'environnement.

1. Créez un modèle d'environnement réutilisable avant de copier les requêtes curl.

Exemple de requête : `export BPM_BASE_URL="${BPM_BASE_URL:?set BPM_BASE_URL first}"`.

Exemple de réponse : le shell conserve une URL de base, un canal de schéma, un nom de profil synthétique et un chemin d'importation facultatif pour toute l'exécution. Exemple d'erreur : le shell quitte l'affectation lorsqu'une valeur requise est manquante.

```
# .env.devops-example.template
BPM_BASE_URL=
BPM_SCHEMA_CHANNEL=release-153
BPM_PROFILE_NAME=docs-api-example
BPM_JSON_IMPORT_PATH=./policies.json

export BPM_BASE_URL="${BPM_BASE_URL:?set BPM_BASE_URL first}"
export BPM_SCHEMA_CHANNEL="${BPM_SCHEMA_CHANNEL:-release-153}"
export BPM_PROFILE_NAME="${BPM_PROFILE_NAME:-docs-api-example}"
export BPM_JSON_IMPORT_PATH="${BPM_JSON_IMPORT_PATH:-./policies.json}"
```

2. Gérez chaque séquence réutilisable avec des sondes d'intégrité et de préparation.

Exemple de requête : `curl -fsS "$BPM_BASE_URL/health"` et `curl -fsS`

`"$BPM_BASE_URL/health/ready"`. Exemple de réponse : `{"status": "ok"}` et

`{"status": "ready", "ready": true}`. Exemple d'erreur : curl quitte une valeur non nulle pour les erreurs de transport ou les réponses non-2xx lorsque `-f` est utilisé.

```
curl -fsS "$BPM_BASE_URL/health"
curl -fsS "$BPM_BASE_URL/health/ready"
```

3. Réutilisez les extraits de cycle de vie du profil pour la création, la liste, la lecture et la mise à jour protégée.

Exemple de requête : POST \$BPM_BASE_URL/api/profiles, GET \$BPM_BASE_URL/api/profiles, GET \$BPM_BASE_URL/api/profiles/\$PROFILE_ID, et PATCH \$BPM_BASE_URL/api/profiles/\$PROFILE_ID. Exemple de réponse : 201 ProfileRead, 200 ProfileRead[], 200 ProfileRead, et un incrémenté revision. Exemples d'erreurs : 400, 404, 409 obsolète expected_revision, ou 422.

```
PROFILE_JSON="$(curl -fsS -H 'Content-Type: application/json' \
-d '{"name":"docs-api-example","schema_version":"release-153","flags":{"DisableTelemetry":true}}' \
"$BPM_BASE_URL/api/profiles")"
PROFILE_ID="$(python -c 'import json,sys; print(json.load(sys.stdin)["id"])' <<<"$PROFILE_JSON")"
PROFILE_REVISION="$(python -c 'import json,sys; print(json.load(sys.stdin)["revision"])'
<<<"$PROFILE_JSON")"

curl -fsS "$BPM_BASE_URL/api/profiles?q=docs-api-example&lifecycle=active&limit=50&offset=0"
curl -fsS "$BPM_BASE_URL/api/profiles/$PROFILE_ID"
curl -fsS -X PATCH -H 'Content-Type: application/json' \
-d '{"flags":
{"DisableTelemetry":true,"BlockAboutConfig":true},"expected_revision":$PROFILE_REVISION}' \
"$BPM_BASE_URL/api/profiles/$PROFILE_ID"
```

4. Réutilisez les extraits de validation et d'importation JSON avec une forme de charge utile explicite.

Exemple de requête : POST \$BPM_BASE_URL/api/validate/release-153 et POST \$BPM_BASE_URL/api/profiles/import/firefox/policies.json. Exemple de réponse : 200 {"ok":true,"profile":"release-153"} et 201 ProfileRead. Exemples d'erreurs : 400, 409, 415, 422, ou 503.

```
curl -fsS -H 'Content-Type: application/json' \
-d '{"document":{"policies":{"DisableTelemetry":true,"BlockAboutConfig":true}}}' \
"$BPM_BASE_URL/api/validate/$BPM_SCHEMA_CHANNEL"

curl -fsS -H 'Content-Type: application/json' \
-d '{"name":"docs-json-import-example","schema_version":"release-153","compliance":{"source":"docs-devops-template"},"document":{"policies":{"DisableTelemetry":true}}}' \
"$BPM_BASE_URL/api/profiles/import/firefox/policies.json"
```

5. Réutilisez les extraits d'importation et d'exportation en plusieurs parties pour un transfert basé sur des fichiers.

Exemple de requête : téléchargement de multipart/form-data vers POST \$BPM_BASE_URL/api/profiles/import/firefox/policies.json suivi de GET \$BPM_BASE_URL/api/export/profiles/\$PROFILE_ID/firefox/policies.json?pretty=1. Exemple de réponse : 201 ProfileRead pour l'importation et 200 application/json contenant le fichier canonique Firefox policies pour l'exportation. Exemple d'erreur : 404 profil manquant ou 422 profil invalide ID.

```
curl -fsS \
-F "name=docs-multipart-import-example" \
-F "schema_version=$BPM_SCHEMA_CHANNEL" \
-F 'compliance={"source":"docs-devops-template"}' \
-F "file=@$BPM_JSON_IMPORT_PATH;type=application/json" \
```

```
"$BPM_BASE_URL/api/profiles/import/firefox/policies.json"

curl -fsS "$BPM_BASE_URL/api/export/profiles/$PROFILE_ID/firefox/policies.json?pretty=1"
```

6. Utilisez un assistant Python compact lorsque l'intégration nécessite des assertions de réponse réutilisables.

Exemple de requête : `session.post(f"{base_url}/api/validate/{schema_channel}", json=payload, timeout=10)`. Exemple de réponse : l'assistant se déclenche immédiatement à moins que le statut et le corps JSON correspondent à l'assertion. Exemple d'erreur : les échecs d'assertion, les exceptions de délai d'attente, les `expected_revision` obsolètes ou les erreurs d'API structurées doivent être enregistrés par le produit d'intégration.

```
import os
import requests

base_url = os.environ["BPM_BASE_URL"].rstrip("/")
schema_channel = os.environ.get("BPM_SCHEMA_CHANNEL", "release-153")
session = requests.Session()

ready = session.get(f"{base_url}/health/ready", timeout=10)
ready.raise_for_status()
assert ready.json() == {"status": "ready", "ready": True}

payload = {"document": {"policies": {"DisableTelemetry": True, "BlockAboutConfig": True}}}
validation = session.post(f"{base_url}/api/validate/{schema_channel}", json=payload, timeout=10)
validation.raise_for_status()
assert validation.json() == {"ok": True, "profile": schema_channel}

created = session.post(
    f"{base_url}/api/profiles",
    json={"name": "docs-python-example", "schema_version": schema_channel, "flags":
payload["document"]["policies"]},
    timeout=10,
)
created.raise_for_status()
profile = created.json()
assert profile["schema_version"] == schema_channel

exported = session.get(
    f"{base_url}/api/export/profiles/{profile['id']}/firefox/policies.json?pretty=1",
    timeout=10,
)
exported.raise_for_status()
assert exported.json() == {"policies": payload["document"]["policies"]}
```

Avertissement :

Conservez l'état de la réponse, le profil ID, la révision, le canal de schéma, le hachage du document exporté et le résultat de l'assertion dans des preuves externes. Ne copiez pas les vrais noms d'hôte, secrets, clés API, valeurs secrètes ou identifiants de production dans des exemples de documentation.

Le produit d'intégration contient des exemples curl et Python réutilisables qui peuvent être copiés en toute sécurité, indépendants des hôtes de déploiement, synchronisés avec le comportement actuel de l'API BPM et liés à des assertions de réponse explicites.

Combinez ces exemples avec les guides d'exploitation d'extraction/liste/lecture, de validation avant application, d'importation-révision-exportation et d'audit d'échec lors de la création de processus de produits de contrôle externes complets.

Synchroniser les données du cycle de vie du profil

Utilisez la liste de profils, les statistiques, lisez, créez et mettez à jour les points de terminaison sans assumer un comportement groupé ou transactionnel.

Définissez `$BPM_BASE_URL` et lisez `/openapi.json` pour les formes `ProfileCreate`, `ProfileUpdate` et `ProfileRead` actuelles.

Cette tâche couvre API-PROFILE-001, API-PROFILE-002, API-PROFILE-003, API-PROFILE-004 et API-PROFILE-005. Le magasin a renvoyé `id` et `revision` ; ne déduisez pas l'identité du seul nom.

1. Répertoirez les profils avec des filtres avant de choisir un profil à synchroniser.

Exemple de requête : `GET $BPM_BASE_URL/api/profiles?`

`q=corp&lifecycle=active&limit=50&offset=0&sort=updated_at&order=desc`. Exemple de

réponse : `200 ProfileRead[]`. Exemple d'erreur : `422` pour une entrée de requête saisie ou étendue non valide.

2. Lisez les statistiques de profil pour le même contexte de filtre lorsque vous avez besoin de décomptes.

Exemple de requête : `GET $BPM_BASE_URL/api/profiles/stats?q=corp&lifecycle=active`.

Exemple de réponse : `200 {"filtered":1,"total":3}`. Exemple d'erreur : `422` pour une entrée de requête non valide.

3. Lit le profil sélectionné par l'entier ID et inclut les enregistrements archivés uniquement lorsque cela est nécessaire.

Exemple de requête : `GET $BPM_BASE_URL/api/profiles/42?include_deleted=false`. Exemple de réponse : `200 ProfileRead` avec `id`, `revision`, `horodatages`, `cycle de vie` et `état de validation`.

Exemple d'erreur : `404` pour un profil manquant ou non visible ; `422` pour un chemin ou une entrée de requête non valide.

4. Créez un profil BPM normalisé uniquement lorsque l'intégration possède le nouveau nom de profil.

Exemple de requête : `POST $BPM_BASE_URL/api/profiles` avec JSON `{"name":"corp-`

`baseline","schema_version":"release-153","flags":{},"compliance":{}}`. Exemple de

réponse : `201 ProfileRead`. Exemples d'erreur : `400` schéma/profil non valide, `409` nom en double ou `422` demande ou validation de stratégie.

5. Mettez à jour les champs mutables avec `PATCH` et envoyez `expected_revision` lorsque vous avez besoin d'une concurrence optimiste.

Exemple de requête : `PATCH $BPM_BASE_URL/api/profiles/42` avec JSON

`{"description":"Reviewed by control plane","expected_revision":7}`. Exemple de réponse :

`200 ProfileRead` avec révision incrémentée après une mise à jour matérielle. Exemples d'erreur :

`400` schéma/profil non valide, `404` profil manquant ou archivé, `409` obsolète `expected_revision` ou erreur de validation `422`.

6. Enregistrez le profil synchronisé ID, la révision, l'état de validation, le canal de schéma et toutes les métadonnées de conformité opaques dans le produit d'intégration.

Avertissement :

BPM ne fournit pas de transaction multi-opérations. Si une étape ultérieure échoue, le produit d'intégration est propriétaire des décisions de récupération, d'audit et de nouvelle tentative.

L'intégration dispose d'un chemin de synchronisation de profil reproductible pour les opérations de liste, de statistiques, de lecture, de création et de mise à jour.

Utilisez la tâche de cycle de vie destructrice avant les opérations d'archivage, de restauration, de suppression définitive ou de réinitialisation.

Archiver, restaurer, supprimer définitivement ou réinitialiser les profils

Utilisez des points de terminaison de cycle de vie de profil destructeurs avec des enregistrements de révision et de récupération explicites.

Confirmez le profil cible ID, l'état actuel du cycle de vie et l'enregistrement d'approbation externe avant d'appeler des points de terminaison destructeurs.

Cette tâche couvre API-PROFILE-006, API-PROFILE-007, API-PROFILE-008 et API-PROFILE-009. BPM n'a pas de jeton de confirmation groupée, pas de point de terminaison d'annulation pour la suppression permanente et aucune annulation de transaction entre les opérations.

1. Archivez un profil lorsque vous devez le supprimer des flux de travail actifs tout en le préservant pour révision.

Avertissement :

Archive est un changement destructeur du cycle de vie des flux de travail actifs. Enregistrez le motif d'approbation et le profil ID avant l'appel.

Exemple de requête : `DELETE $BPM_BASE_URL/api/profiles/42`. Exemple de réponse : 204 sans corps JSON. Exemples d'erreur : 404 profil manquant ou 422 entrée de chemin non valide.

2. Restaurez un profil archivé uniquement après avoir vérifié si le nom et l'objectif du profil sont toujours valides.

Exemple de requête : `POST $BPM_BASE_URL/api/profiles/42/restore`. Exemple de réponse : 200 ProfileRead. Exemples d'erreur : le profil 404 ne peut pas être restauré ou trouvé ; 422 entrée de chemin non valide.

3. Supprimez définitivement un profil uniquement lorsque la stratégie de rétention externe autorise la suppression.

Avertissement :

La suppression permanente supprime un profil actif ou archivé et n'a pas de point de terminaison d'annulation BPM. Exportez ou enregistrez les preuves requises avant l'appel.

Exemple de requête : `DELETE $BPM_BASE_URL/api/profiles/42/hard`. Exemple de réponse : 204 sans corps JSON. Exemples d'erreur : 404 profil manquant ou 422 entrée de chemin non valide.

4. Réinitialisez la bibliothèque uniquement dans des environnements jetables ou explicitement approuvés.

Avertissement :

La réinitialisation supprime définitivement tous les profils de la bibliothèque et ne nécessite actuellement aucun jeton de confirmation. Ne l'utilisez jamais comme raccourci de synchronisation de routine.

Exemple de requête : DELETE \$BPM_BASE_URL/api/profiles/reset. Exemple de réponse : 200 {"deleted":3}. Comportement d'erreur : aucune erreur d'application n'est actuellement déclarée ; les échecs de base de données utilisent la gestion du framework.

5. Après tout appel destructeur, réénumérez les profils et lisez l'enregistrement concerné, le cas échéant.

Exemple de requête : GET \$BPM_BASE_URL/api/profiles?lifecycle=all. Exemple de réponse : 200 ProfileRead[]. Exemple d'erreur : 422 entrée de requête non valide. Utilisez cette vérification pour mettre à jour l'état de l'inventaire du produit d'intégration.

6. Écrivez l'enregistrement d'audit externe avec l'opération ID, la méthode HTTP, le chemin, le profil ID, l'approbateur, le motif, l'horodatage, l'état de la réponse et la décision de récupération.

Résultat attendu : un réviseur ultérieur peut distinguer les actions d'archivage, de restauration, de suppression matérielle et de réinitialisation et peut voir pourquoi l'intégration a considéré l'opération comme sûre.

Les opérations destructives du cycle de vie des profils sont documentées avec des preuves de demande, de réponse, d'erreur, d'avertissement et de récupération.

Si l'enregistrement d'approbation est manquant, arrêtez le processus d'intégration avant d'appeler la suppression ou la réinitialisation permanente.

Importer Firefox policies.json via l'API

Créez un profil BPM normalisé à partir d'un document canonique Firefox Enterprise policies.json à l'aide d'une entrée JSON ou multipart.

Définissez \$BPM_BASE_URL, choisissez un canal de schéma pris en charge tel que release-153 et préparez un document Firefox complet avec un objet policies de niveau supérieur.

Cette tâche couvre API-FF-001. L'importation accepte application/json avec FirefoxPoliciesJsonImportRequest ou multipart/form-data avec un champ file. BPM valide le document canonique Firefox, normalise policies dans le profil flags et stocke les métadonnées opaques facultatives compliance sur le profil BPM.

1. Soumettez une demande d'importation JSON lorsque le produit intégrateur a déjà le document en mémoire.

Exemple de requête : POST \$BPM_BASE_URL/api/profiles/import/firefox/policies.json avec Content-Type: application/json et JSON {"name":"api-json-import","description":"Imported by control product","schema_version":"release-153","compliance":{"framework":"cis","layer":"cis_l1"},"document":{"policies":{"DisableTelemetry":true,"BlockAboutConfig":true}}}.

2. Utilisez l'importation partitionnée lorsque le système source remet un fichier.

Exemple de requête : POST \$BPM_BASE_URL/api/profiles/import/firefox/policies.json avec multipart/form-data, champs name=api-multipart-import, schema_version=release-153, description facultatif, compliance={"framework":"cis","layer":"cis_l2"} facultatif sous forme de chaîne JSON et file=@policies.json;type=application/json. Le runtime accepte également un champ multipart document, mais le schéma multipart OpenAPI déclare actuellement file ; utilisez file pour de nouveaux exemples.

3. Lisez le profil BPM créé à partir de la réponse 201 ProfileRead.

Exemple de réponse : 201 ProfileRead avec schema_version="release-153", flags.DisableTelemetry=true, flags.BlockAboutConfig=true, stockés compliance, id et revision. Stockez le profil renvoyé ID au lieu de déduire l'identité à partir du nom du profil.

4. Traitez les erreurs d'importation comme des échecs de création de profil et ne supposez pas un état de profil partiel.

Exemples d'erreurs : 400 pour un JSON mal formé, une forme de document incorrecte ou un canal de schéma non pris en charge ; 409 pour le nom de profil en double ; 415 pour un type de média non pris en charge ; 422 pour les erreurs de demande ou de validation de la politique Firefox.

5. Gardez Firefox canonique et les formes BPM normalisées séparées dans le code d'intégration.

La requête document.policies est la limite de déploiement de Firefox. Les flags du profil créé sont le modèle normalisé de BPM. N'envoyez pas d'objet flags de niveau supérieur à ce point de terminaison d'importation.

6. Enregistrez la preuve d'importation dans le produit d'intégration.

Avertissement :

L'importation crée un profil BPM mais ne déploie pas les paramètres de Firefox, ne prouve pas l'application de l'exécution ou ne fournit pas de clé d'idempotence. Enregistrez le hachage du document source, le canal de schéma, le profil renvoyé ID, la révision, les métadonnées de conformité et la décision de récupération de nom en double en externe.

Le document Firefox policies.json est représenté comme un profil BPM normalisé qui peut être examiné, validé, exporté et suivi par ID.

Utilisez la tâche d'exportation pour restituer le profil enregistré au format Firefox JSON canonique avant de le transmettre aux outils de déploiement.

Exporter un fichier Firefox policies.json canonique via l'API

Affichez un profil BPM en tant que document de déploiement de Firefox Enterprise avec des paramètres de téléchargement et d'impression facultatifs.

Stocke le profil BPM ID renvoyé par création ou importation. Si le profil est archivé, décidez explicitement si l'intégration est autorisée à l'exporter avec `include_deleted=true`.

Cette tâche couvre API-FF-002. L'exportation lit le flags normalisé de BPM et renvoie le JSON canonique de Firefox avec un objet policies de niveau supérieur. Les métadonnées du profil BPM et les métadonnées opaques compliance ne sont pas exportées dans le document de déploiement de Firefox.

1. Exportez le profil actif au format Firefox JSON canonique.

Exemple de requête : `GET $BPM_BASE_URL/api/export/profiles/42/firefox/policies.json`.

Exemple de réponse : 200 application/json avec JSON `{"policies":`

`{"DisableTelemetry":true,"BlockAboutConfig":true}}`.

2. Ajoutez le mode de téléchargement lorsque l'appelant a besoin d'un nom de fichier de pièce jointe.

Exemple de requête : `GET $BPM_BASE_URL/api/export/profiles/42/firefox/policies.json?`

`download=1`. Exemple de réponse : 200 application/json plus Content-Disposition:

`attachment; filename="profile-42-policies.json"`. Le corps JSON est le même document de déploiement.

3. Choisissez délibérément les paramètres de formatage.

Exemple de requête : `GET $BPM_BASE_URL/api/export/profiles/42/firefox/policies.json?`

`pretty=1` sélectionne l'indentation de deux espaces lorsque indent est omis. Exemple de requête :

`GET $BPM_BASE_URL/api/export/profiles/42/firefox/policies.json?indent=0` utilise le

paramètre d'indentation explicite. Exemple d'erreur : 422 pour des valeurs de requête contrainte non valides.

4. Exportez un profil archivé uniquement lorsque le processus externe a un motif de révision.

Exemple de requête : `GET $BPM_BASE_URL/api/export/profiles/42/firefox/policies.json?`

`include_deleted=true&download=1&pretty=1`. Exemple de réponse : 200 application/json.

Exemples d'erreur : 404 lorsqu'un profil manquant ou archivé n'est pas visible ; 422 pour un chemin ou une entrée de requête non valide.

5. Validez le document exporté avant de le transmettre à un autre système.

Exemple de requête : `POST $BPM_BASE_URL/api/validate/release-153` avec JSON `{"document":`

`{"policies":{"DisableTelemetry":true,"BlockAboutConfig":true}}}`. Exemple de réponse :

200 `{"ok":true,"profile":"release-153"}`. La validation confirme la forme du schéma, et non le succès du déploiement du runtime Firefox.

6. Enregistrez l'artefact exporté exact en externe.

Avertissement :

L'exportation n'a pas de ETag, d'en-tête de précondition, de point de terminaison groupé ou de wrapper de transaction. Enregistrez le profil ID, la révision observée avant l'exportation, le canal de schéma, les paramètres de requête, le hachage du document exporté et la décision de transfert en aval en dehors de BPM.

L'intégration possède un artefact canonique `Firefox policies.json` qui exclut les métadonnées BPM uniquement et peut être fourni aux outils de déploiement.

Si la validation échoue ou si la révision du profil a été modifiée entre la lecture et l'exportation, arrêtez le transfert et actualisez l'état du profil.

Valider un fichier Firefox policies.json candidat via l'API

Utilisez le point de terminaison de validation comme porte de contrôle en amont pour les documents candidats Firefox Enterprise policies.json et la gestion structurée des erreurs.

Choisissez le canal de schéma cible, tel que release-153 ou esr-140.13, et préparez un corps ValidationRequest avec un document Firefox complet dans document.policies.

Cette tâche couvre API-VAL-001. BPM valide le contenu candidat policies.json avant l'importation, le transfert d'exportation ou une porte de révision de stratégie externe. Un mappage de politique simple est toujours accepté pour des raisons de compatibilité, mais les nouveaux exemples d'intégration utilisent la forme Firefox policies.json complète.

1. Soumettez une demande de validation réussie pour le canal de schéma sélectionné.

Exemple de requête : POST \$BPM_BASE_URL/api/validate/release-153 avec JSON {"document":{"policies":{"DisableTelemetry":true,"BlockAboutConfig":true}}}. Exemple de réponse : 200 {"ok":true,"profile":"release-153"}.

2. Traitez ok=false comme une porte défaillante même lorsque l'état de HTTP est 200.

Exemple de requête : POST \$BPM_BASE_URL/api/validate/release-153 avec JSON {"document":123}. Exemple de réponse : 200 {"ok":false,"profile":"release-153","detail":"Expected object with policy mappings","error":"Expected object with policy mappings"}. Vérifiez le corps de la réponse, pas uniquement le statut HTTP.

3. Gérer les erreurs de documents Firefox canoniques mal formés en tant qu'échecs de forme de demande.

Exemple de requête : POST \$BPM_BASE_URL/api/validate/release-153 avec JSON {"document":{"policies":[]}}. Exemple d'erreur : 400 avec detail.message="Firefox policies.json validation failed" et detail.issues[].path pointant vers policies.

4. Rejetez les canaux de schéma non pris en charge avant de faire confiance aux résultats de validation.

Exemple de requête : POST \$BPM_BASE_URL/api/validate/beta-999 avec JSON {"document":{"policies":{"DisableTelemetry":true}}}. Exemple d'erreur : 404 avec le texte détaillé Unknown profile 'beta-999'. Un schéma enregistré mais indisponible renvoie 503.

5. Mappez les échecs de validation des politiques aux commentaires exploitables des évaluateurs.

Exemple de requête : POST \$BPM_BASE_URL/api/validate/release-153 avec JSON {"document":{"policies":{"Proxy":{"Mode":"bogus"}}}}. Exemple d'erreur : 422 avec detail.message="Policy validation failed" et champs de problème policy, path et message.

6. Enregistrez la décision de validation dans le produit intégrateur.

Avertissement :

La validation est uniquement une porte de forme de schéma et de politique. Il ne déploie pas Firefox, ne prouve pas l'application de l'exécution, n'authentifie pas les appelants, ne réserve pas de révision de profil et ne fournit pas de clé d'idempotence. Stockez le canal, le hachage du document candidat, l'état de la réponse, la valeur ok, les problèmes structurés et la décision du réviseur en externe.

L'intégration peut bloquer les transferts de candidats non sécurisés policies.json et préserver des preuves structurées pour les décisions de réussite et d'échec.

Si la validation réussit, continuez l'importation, la mise à jour du profil ou l'exportation du transfert en utilisant le même canal de schéma et le même hachage candidat enregistré.

Exécuter les flux de cycle de vie et rétablir le service

Exécuter un scénario d'extraction, de comparaison et de mise à jour du produit de contrôle

Utilisez le profil existant et la validation API pour extraire l'état du BPM, le comparer en externe et mettre à jour un profil avec des preuves explicites de propriété et de concurrence.

Définissez \$BPM_BASE_URL et décidez quel produit de contrôle externe possède la logique de comparaison, le calcul de l'état souhaité, la stratégie de nouvelle tentative et les enregistrements d'audit. BPM n'expose pas d'API de comparaison ni de wrapper de transaction.

Ce scénario utilise API-PROFILE-001, API-PROFILE-003, API-PROFILE-005 et API-VAL-001. BPM possède l'état et la validation du profil stocké. Le produit d'intégration possède l'état souhaité, le résultat de la comparaison, l'approbation du réviseur et les décisions de nouvelle tentative.

1. Extrayez les profils de candidats de la bibliothèque avec les mêmes règles de filtrage que celles utilisées par le produit de contrôle pour la propriété.

Exemple de requête : GET \$BPM_BASE_URL/api/profiles?
q=corp&lifecycle=active&limit=50&offset=0. Exemple de réponse : 200 ProfileRead[].
Exemple d'erreur : 422 pour une entrée de requête non valide.

2. Lit chaque profil sélectionné par ID et stocke son revision actuel.

Exemple de requête : GET \$BPM_BASE_URL/api/profiles/42. Exemple de réponse : 200
ProfileRead avec flags, compliance, schema_version et revision. Exemples d'erreur : 404 profil
manquant ou archivé ; 422 entrée de chemin non valide.

3. Comparez le profil extrait avec l'état externe souhaité en dehors de BPM.

Résultat attendu : le produit intégrateur enregistre les clés de stratégie modifiées, les clés
inchangées, les clés manquantes, le réviseur et la source de l'état souhaité. Ne présumez pas qu'il
existe un point de terminaison de comparaison BPM, un connecteur Marketplace ou un processus
certifié.

4. Validez le document Firefox candidat avant d'appliquer la mise à jour.

Exemple de requête : POST \$BPM_BASE_URL/api/validate/release-153 avec JSON {"document":
{"policies":{"DisableTelemetry":true,"BlockAboutConfig":true}}}. Exemple de réponse :
200 {"ok":true,"profile":"release-153"}. Exemples d'erreur : 400, 404, 422 ou 503 en fonction
de la forme du document, du canal, de la validité de la stratégie et de la disponibilité du schéma.

5. Corrigez le profil BPM uniquement avec la révision observée avant la comparaison.

Exemple de requête : PATCH \$BPM_BASE_URL/api/profiles/42 avec JSON {"flags":
{"DisableTelemetry":true,"BlockAboutConfig":true},"expected_revision":7,"compliance":
{"source":"control-product","decision":"approved"}}. Exemple de réponse : 200
ProfileRead avec revision incrémenté. Exemples d'erreurs : 400, 404, 409 obsolète
expected_revision ou 422.

6. Gérer les échecs sans relecture automatique.

Avertissement :

Si la validation échoue, que la révision est obsolète ou que le résultat de la mise à jour ne correspond pas à l'état souhaité, arrêtez le flux de travail, relisez l'état du BPM et créez un enregistrement d'audit externe. BPM ne fournit pas de transaction multi-opérations, de restauration automatique, de clé d'idempotence ou de contrat de nouvelle tentative sécurisée.

Le produit de contrôle dispose d'un flux d'extraction/comparaison/mise à jour exécutable avec propriété d'état explicite, preuves de validation, protection contre les révisions et limites de reprise après incident.

Si le profil mis à jour doit être déployé ailleurs, poursuivez la tâche d'exportation Firefox et conservez le profil ID, la révision, la réponse de validation et l'enregistrement d'approbation.

Exécuter un scénario d'importation, de révision, d'exportation et de transfert de conformité

Utilisez l'état de santé, l'importation, la validation, le profil et l'exportation API existants pour transmettre un document Firefox révisé avec des preuves de conformité.

Définissez \$BPM_BASE_URL, choisissez le canal de schéma et décidez quel système possède le document source, le résultat de l'analyse de conformité, l'approbation du réviseur, l'artefact exporté et la décision de récupération.

Ce scénario utilise API-HEALTH-001, API-HEALTH-002, API-FF-001, API-PROFILE-003, API-VAL-001 et API-FF-002. BPM stocke le profil normalisé et les métadonnées de conformité opaques ; le produit intégrateur possède l'interprétation de l'analyse et le déploiement en aval.

1. Commencez par la santé et la préparation avant la séquence de transfert.

Exemples de requête : GET \$BPM_BASE_URL/health et GET \$BPM_BASE_URL/health/ready. Exemples de réponses : 200 {"status":"ok"} et 200 {"status":"ready", "ready":true}. Exemple d'erreur : l'échec du transport ou de la passerelle est géré en dehors de BPM car aucune erreur d'application n'est actuellement déclarée.

2. Importez le document Firefox révisé avec les métadonnées de conformité attachées au profil BPM.

Exemple de requête : POST \$BPM_BASE_URL/api/profiles/import/firefox/policies.json avec JSON {"name":"reviewed-corp-baseline", "schema_version":"release-153", "compliance":{"scanner":"external-control", "result":"passed", "ticket":"SEC-42"}, "document":{"policies":{"DisableTelemetry":true, "BlockAboutConfig":true}}}. Exemple de réponse : 201 ProfileRead. Exemples d'erreurs : 400, 409, 415, ou 422.

3. Lisez le profil créé et enregistrez les champs de propriété BPM séparément des preuves externes.

Exemple de requête : GET \$BPM_BASE_URL/api/profiles/42. Exemple de réponse : 200 ProfileRead avec id, revision, flags, schema_version et compliance opaque. Exemples d'erreur : 404 ou 422.

4. Validez le document canonique Firefox qui sera transmis.

Exemple de requête : POST \$BPM_BASE_URL/api/validate/release-153 avec JSON {"document":{"policies":{"DisableTelemetry":true, "BlockAboutConfig":true}}}. Exemple de réponse : 200 {"ok":true, "profile":"release-153"}. Exemples d'erreurs : 400, 404, 422, ou 503.

5. Exportez le profil BPM en tant que document de déploiement canonique de Firefox.

Exemple de requête : GET \$BPM_BASE_URL/api/export/profiles/42/firefox/policies.json?download=1&pretty=1. Exemple de réponse : 200 application/json avec Content-Disposition et policies de niveau supérieur. Les métadonnées du profil BPM et les métadonnées de conformité ne sont pas exportées.

6. Arrêtez-vous en cas de non-concordance et écrivez l'enregistrement de reprise après incident.

Avertissement :

Si les preuves d'importation, de validation, de relecture ou d'exportation diffèrent du document source approuvé, ne réessayez pas aveuglément. Stockez le hachage source, le profil ID, la révision, le corps de validation, le hachage d'exportation, le résultat de l'analyseur, le réviseur, l'état de la réponse et la décision de récupération en dehors de BPM.

Le produit d'intégration dispose d'un flux d'importation-révision-exportation exécutable avec transfert de conformité, exportation canonique et preuve de récupération explicite.

Utiliser des outils de déploiement en dehors de BPM pour appliquer le `policies.json` exporté ; La validation BPM ne prouve pas l'application du runtime Firefox.

Extraire et lire les profils BPM à partir d'un produit de contrôle externe

Créez le côté pull/list/lecture d'une intégration contrôle-produit avec une propriété explicite, des filtres et des limites de preuves.

Définissez `$BPM_BASE_URL`. Décider quel produit de contrôle externe est propriétaire de la sélection des stocks, du stockage comparatif, de la politique de nouvelle tentative et des éléments probants ; BPM est propriétaire uniquement des enregistrements de profil renvoyés par son API.

Ce flux de travail utilise API-PROFILE-001, API-PROFILE-002 et API-PROFILE-003. Il s'agit du point d'entrée sécurisé en lecture seule avant les processus de comparaison/mise à jour, de validation avant application, d'importation-révision-exportation ou de transfert de métadonnées de conformité.

1. Confirmez la portée du produit de contrôle avant de lire l'état du BPM.

Enregistrez le propriétaire externe, l'objectif, le canal de schéma, les règles de filtrage et si les profils archivés sont exclus. Utilisez `lifecycle=active` sauf si une procédure de récupération nécessite explicitement `include_deleted=true`.

2. Répertoirez les profils avec des filtres déterministes et une pagination.

Exemple de requête : `GET $BPM_BASE_URL/api/profiles?q=corp&lifecycle=active&limit=50&offset=0&sort=updated_at&order=desc`. Exemple de réponse : `200 ProfileRead[]`. Exemple d'erreur : `422` pour des paramètres de requête non valides.

3. Capturez les inventaires facultatifs séparément des données de profil.

Exemple de requête : `GET $BPM_BASE_URL/api/profiles/stats?q=corp&lifecycle=active`.
Exemple de réponse : `200 application/json`. Stockez les preuves de décompte en tant que contexte d'inventaire consultatif, et non en tant que verrou ou limite de transaction.

4. Lisez chaque profil sélectionné par ID avant comparaison.

Exemple de requête : `GET $BPM_BASE_URL/api/profiles/42?include_deleted=false`. Exemple de réponse : `200 ProfileRead` avec `id`, `revision`, `schema_version`, `flags` et `compliance`. Exemples d'erreur : `404` ou `422`.

5. Conserver un instantané de lecture en dehors de BPM pour le produit de contrôle.

Stockez le profil ID, la révision, le canal de schéma, la requête de filtre, l'horodatage de lecture, l'URL source, les clés de stratégie sélectionnées et l'exécution du produit de contrôle ID. Ne stockez pas de secrets ou de noms d'hôtes privés dans des exemples de documentation partagée.

6. Arrêtez-vous si le résultat de l'extraction n'est pas fiable.

Avertissement :

Si la pagination, le filtrage ou la relecture des preuves sont incomplètes, arrêtez-vous avant la mise à jour. BPM n'expose pas la pagination du curseur, l'isolation des instantanés, les verrous de comparaison, les webhooks ou un contrat de relecture sécurisé pour les flux de travail de lecture.

Le produit de contrôle externe dispose d'un instantané d'extraction/liste/lecture reproductible et sait quelles preuves appartiennent au BPM et quelles preuves appartiennent au système externe.

Continuez avec la validation avant application ou la comparaison/mise à jour uniquement après que le système externe a enregistré le profil sélectionné identifiants et les révisions.

Valider et appliquer une mise à jour de profil avec `expected_revision`

Exécutez un processus de mise à jour de validation qui protège l'état du BPM avec la révision actuelle et s'arrête sur les données obsolètes ou non valides.

Le produit externe a déjà lu le profil cible et stocké son `revision` actuel. Préparez le document candidat `Firefox policies.json` avant d'envoyer un correctif de profil.

Ce flux de travail utilise `API-VAL-001`, `API-PROFILE-003` et `API-PROFILE-005`. Il documente le chemin de mise à jour optimiste actuel ; BPM ne fournit pas de ETags, d'en-têtes de préconditions, de clés d'idempotence ou de transactions multi-opérations.

1. Créez le document candidat Firefox en dehors de BPM.

Enregistrez les clés de stratégie souhaitées, le système source, le réviseur, le ticket et le canal de schéma. Exemple de candidat : `{"document":{"policies":{"DisableTelemetry":true,"BlockAboutConfig":true}}}`.

2. Validez le candidat avant de corriger le profil.

Exemple de requête : `POST $BPM_BASE_URL/api/validate/release-153`. Exemple de réponse : `200 {"ok":true,"profile":"release-153"}`. Exemples d'erreur : 400, 404, 422 ou 503.

3. Relisez le profil immédiatement avant d'appliquer la mise à jour lorsque la fenêtre d'approbation est longue.

Exemple de requête : `GET $BPM_BASE_URL/api/profiles/42`. Exemple de réponse : `200 ProfileRead`. Si `revision` a changé, réexécutez la comparaison et l'approbation plutôt que de corriger l'état obsolète.

4. Corrigez uniquement les champs mutables avec la révision observée.

Exemple de requête : `PATCH $BPM_BASE_URL/api/profiles/42` avec JSON `{"flags":{"DisableTelemetry":true,"BlockAboutConfig":true},"expected_revision":7,"compliance":{"source":"control-product","decision":"approved"}}`. Exemple de réponse : `200 ProfileRead` avec `revision` incrémenté.

5. Traitez les réponses conflictuelles comme des signaux d'arrêt.

Exemples d'erreurs : 400 charge utile non valide, 404 profil manquant, 409 obsolète `expected_revision` et 422 erreurs de requête au niveau du schéma. Un 409 signifie que l'état externe souhaité doit être recalculé.

6. Enregistrez les preuves de mise à jour et évitez les rediffusions aveugles.

Avertissement :

Ne réessayez pas aveuglément le même `PATCH`. Stockez l'ancienne révision, la nouvelle révision, le corps de validation, le hachage du corps du correctif, l'état de la réponse, l'approbation du réviseur et la décision de nouvelle tentative en dehors de BPM.

La mise à jour du profil est appliquée uniquement après des contrôles de validation et de révision, avec des preuves d'audit explicites de réussite ou de conflit.

Exportez le profil ou transférez les métadonnées de conformité uniquement une fois que le profil de relecture correspond à l'état souhaité approuvé.

Exécuter le transfert des métadonnées d'importation, de révision, d'exportation et de conformité

Connectez un produit de révision ou de conformité externe à l'importation et à l'exportation BPM API sans traiter les métadonnées BPM comme du contenu de déploiement Firefox.

Définissez `$BPM_BASE_URL` et préparez un document source Firefox `policies.json` révisé ainsi que des métadonnées de conformité externes telles que le scanner, le ticket, le réviseur et la décision.

Ce flux de travail utilise API-FF-001, API-PROFILE-003, API-VAL-001, et API-FF-002. BPM stocke des métadonnées de conformité opaques sur le profil, mais Firefox JSON exporté ne contient que des métadonnées déployables `policies`.

1. Importez le document source révisé dans un profil BPM.

Exemple de requête : `POST $BPM_BASE_URL/api/profiles/import/firefox/policies.json` avec `JSON {"name":"reviewed-corp-baseline","schema_version":"release-153","compliance":{"scanner":"external-control","ticket":"SEC-42"},"document":{"policies":{"DisableTelemetry":true}}}`. Exemple de réponse : `201 ProfileRead`.

2. Relisez le profil créé et séparez les champs BPM des preuves externes.

Exemple de requête : `GET $BPM_BASE_URL/api/profiles/42`. Exemple de réponse : `200 ProfileRead` avec `revision`, `flags` et `compliance` opaque. Exemples d'erreur : `404` ou `422`.

3. Vérifiez l'ensemble de règles normalisées avant l'exportation.

Le réviseur vérifie les clés importées, le canal de schéma, le hachage du document source, les métadonnées de conformité et tous les avertissements de normalisation BPM. BPM ne certifie pas le résultat de l'analyseur externe ni l'application du runtime Firefox.

4. Validez le document de déploiement qui sera exporté.

Exemple de requête : `POST $BPM_BASE_URL/api/validate/release-153` avec `JSON {"document":{"policies":{"DisableTelemetry":true}}}`. Exemple de réponse : `200 {"ok":true}`. Exemples d'erreurs : `400`, `404`, `422`, ou `503`.

5. Exportez le document de politiques canoniques de Firefox.

Exemple de requête : `GET $BPM_BASE_URL/api/export/profiles/42/firefox/policies.json?download=1&pretty=1`. Exemple de réponse : `200 application/json` avec `Content-Disposition` et `policies` de niveau supérieur. Les métadonnées de conformité ne sont pas exportées.

6. Écrivez l'enregistrement de transfert avant le déploiement en aval.

Avertissement :

Stocke le hachage source, le profil ID, la révision, la réponse de validation, le hachage d'exportation, le résultat de l'analyseur, le réviseur et le propriétaire en aval en dehors de BPM. Ne considérez pas l'exportation BPM comme une preuve que Firefox a appliqué la politique.

Le produit externe reçoit un document de déploiement canonique de Firefox et un enregistrement de transfert de conformité distinct avec des limites de propriété claires.

Utilisez des outils de déploiement en dehors de BPM pour placer le `policiers.json` exporté sur les systèmes Firefox gérés.

Démarrer le produit de contrôle après vérification de l'intégrité et de l'état de préparation de BPM

Utilisez les sondes d'intégrité actuelles comme porte de démarrage avant qu'un produit de contrôle externe ne commence la synchronisation de l'API.

Le déploiement de la source BPM est déjà démarré. Configurez le produit externe avec `$BPM_BASE_URL`, le délai d'attente, l'intervalle d'interrogation, l'attente maximale et l'action à entreprendre en cas de défaillance de la porte.

Ce flux de travail utilise API-HEALTH-001 et API-HEALTH-002. Les sondes sont uniquement de petits signaux de prévol ; ils n'exposent pas l'authentification, l'autorisation, les diagnostics de base de données, les diagnostics de cache de schéma, la requête identifiants ou les objectifs de niveau de service. Cet ensemble de guides d'exploitation nomme le démarrage sécurisé, la propriété des données, la récupération après échec et la règle de ne pas réessayer aveuglément lorsque les preuves sont périmées.

La disponibilité de l'assistant de documentation est distincte de l'intégrité et de l'état de préparation du cœur de BPM. Dans la version 0.9.3, chaque question à l'assistant reçoit l'avis publié sur l'apprentissage du modèle local ; aucun modèle, aucune recherche, aucun index RAG ni aucune requête vers des sources externes ne démarre. Cette limite ne bloque pas la recherche déterministe et ne doit pas servir de porte aux modifications de profils ni à la synchronisation d'API.

1. Vérifiez l'activité de base avant d'ouvrir la file d'attente de synchronisation.

Exemple de requête : `GET $BPM_BASE_URL/health`. Exemple de réponse : `200 {"status": "ok"}`. Les erreurs de transport sont gérées par le produit externe car aucun corps d'erreur d'application BPM n'est déclaré pour cette sonde.

2. Vérifiez l'état de préparation avant les appels de profil ou de validation.

Exemple de requête : `GET $BPM_BASE_URL/health/ready`. Exemple de réponse : `200 {"status": "ready", "ready": true}`. Si `ready` n'est pas `true`, maintenez la file d'attente et ne démarrez pas les mutations.

3. Enregistrez la décision de la porte de démarrage.

Stockez l'horodatage, l'étiquette de l'URL de base, le délai d'attente, l'état de la sonde, le corps observé, la version du produit de contrôle et l'action suivante choisie. Ne stockez pas d'hôtes privés dans des exemples de runbook partagés.

4. Commencez par un inventaire en lecture seule une fois la porte passée.

Appelez `GET $BPM_BASE_URL/api/profiles?limit=50` avant les opérations d'importation, d'exportation, de validation ou de correctif. Cela permet de conserver la synchronisation initiale observable et récupérable.

5. Passez aux processus de mutation uniquement après la réussite des appels en lecture seule.

Continuez à valider avant application, importer-réviser-exporter ou exporter les flux de travail en fonction du calendrier du produit externe. Conservez la politique de nouvelle tentative et d'attente

en dehors de BPM.

6. Échec de fermeture lorsque le portail est ambigu.

Avertissement :

Une vérification réussie ne réserve pas de révisions et ne garantit pas le succès ultérieur de l'API. Une sonde défaillante ou ambiguë doit arrêter les mutations automatisées jusqu'à ce qu'un opérateur vérifie les journaux, l'état de déploiement de la source et les limites du réseau/CORS.

Le produit de contrôle externe démarre la synchronisation uniquement lorsque BPM répond à la négociation d'intégrité et de préparation actuelle.

Si la porte de démarrage échoue, utilisez les rubriques de dépannage et de déploiement de source Administrateur/DevOps plutôt que d'ajouter des hypothèses de sonde non documentées.

Récupérer des échecs d'intégration et enregistrer les preuves d'audit

Gérez les échecs de validation, d'importation/exportation, de préparation et de révision sans tentatives dangereuses ni hypothèses de transaction cachées.

Utilisez ce flux de travail lorsqu'un produit externe reçoit une réponse d'API BPM ayant échoué, un délai d'attente, un corps de réponse inattendu, un `expected_revision` obsolète ou une preuve d'exportation incompatible.

Ce processus connecte API-HEALTH-001, API-HEALTH-002, API-PROFILE-003, API-PROFILE-005, API-FF-001, API-FF-002 et API-VAL-001. Il définit la preuve de l'opérateur ; BPM ne fournit pas de suivi distribué, de restauration automatique, de récupération de transactions groupées ou de notifications de webhook.

1. Classifiez l'échec avant de réessayer.

Enregistrez si l'échec est lié à l'intégrité/préparation, à la validation, à l'importation, à l'exportation, à la lecture, au correctif, au conflit, au délai d'attente ou au transport. Conservez l'état HTTP tel que 400, 404, 409, 415, 422 ou 503.

2. Geler l'état externe souhaité et l'observation BPM.

Stocker la méthode de requête, le chemin, le hachage de charge utile nettoyé, le profil ID, le `revision` observé, le canal de schéma, le corps de la réponse, le temps de réponse et l'exécution du produit de contrôle ID. N'incluez jamais de secrets ou de noms d'hôtes privés dans les preuves partagées.

3. Vérifiez à nouveau l'état du BPM et relisez le profil lorsque l'échec est récupérable.

Exemples de requête : GET `$BPM_BASE_URL/health`, GET `$BPM_BASE_URL/health/ready` et GET `$BPM_BASE_URL/api/profiles/42`. Si la révision du profil a changé, revenez à la comparaison et à l'approbation.

4. Validez ou exportez à nouveau uniquement après une décision approuvée par l'opérateur.

Pour validation, répéter POST `$BPM_BASE_URL/api/validate/release-153` avec le même document candidat. Pour l'exportation, répétez GET `$BPM_BASE_URL/api/export/profiles/42/firefox/policies.json?pretty=1` et comparez le hachage du document exporté.

5. Arrêtez les tentatives de mutation en cas de révision obsolète ou de preuves ambiguës.

Si `expected_revision` renvoie 409, si le hachage d'exportation diffère ou si les erreurs de validation ont changé, ne poursuivez pas l'ancienne exécution. Ouvrez une nouvelle exécution externe avec une nouvelle lecture, comparaison et approbation du réviseur.

6. Publier l'enregistrement d'audit dans le système d'enregistrement externe.

Avertissement :

BPM stocke l'état du profil mais ne stocke pas une piste d'audit d'intégration complète. Le produit externe doit enregistrer le propriétaire, la décision, le ticket, le nombre de nouvelles tentatives, le motif de l'arrêt, les anciennes et nouvelles révisions, les hachages et les notes de l'opérateur.

Les échecs ne peuvent être récupérés que grâce à des preuves explicites, de nouvelles lectures et des décisions de nouvelle tentative approuvées par l'opérateur, et non par une relecture aveugle.

Après la récupération, liez l'enregistrement d'audit à la révision de profil réussie ou marquez l'exécution de l'intégration comme arrêtée.

Résoudre les problèmes de BPM et évaluer la préparation à la production

Résoudre les problèmes de démarrage échoué et les sondes inaccessibles

Diagnostiquer un processus BPM exécuté par la source qui ne démarre pas ou ne répond pas aux sondes d'intégrité et de préparation.

Utilisez cette procédure lorsque `make dev`, `uvicorn app.main:app --reload --port 8000`, `GET /health` ou `GET /health/ready` échouent lors du déploiement de la source ou du démarrage de l'intégration.

Ce diagnostic sépare les preuves de processus, d'environnement et de sonde. Il ne crée pas d'unité `systemd`, de service Windows, de proxy inverse, de point de terminaison TLS ou de stratégie de redémarrage de la production.

1. Capturez la commande défailante et le journal de premier plan.

Exécutez ou copiez la commande exacte, puis enregistrez `stdout` et `stderr` à partir du terminal de premier plan. Incluez `make dev`, `uvicorn app.main:app --reload --port 8000`, le résumé du traçage Python, le répertoire de travail et la révision source de `git rev-parse --short HEAD`.

2. Vérifiez l'hôte et le port configurés avant de modifier le code.

Vérifiez `BPM_HOST`, `BPM_PORT`, `BPM_RELOAD`, `BPM_DATABASE_URL` et `BPM_DOCUMENTATION_SITE_DIR`. Réexécution ciblée : `curl -fsS http://127.0.0.1:8000/health` et `curl -fsS http://127.0.0.1:8000/health/ready`.

3. Séparez le démarrage du processus de l'accessibilité de la sonde.

Si le processus n'est pas en cours d'exécution, corrigez d'abord les preuves de dépendance, d'environnement ou de migration. Si le processus est en cours d'exécution mais que les sondes sont inaccessibles, vérifiez le pare-feu local, l'adresse de liaison, la redirection de port WSL et si la requête cible `127.0.0.1`, `0.0.0.0` ou une route hôte Windows.

4. Préservez les données des utilisateurs tout en collectant des preuves.

Copiez uniquement les journaux, les noms d'environnement et les réponses de sonde. Ne modifiez pas les artefacts générés, les lignes de base de données, les fichiers de cache de schéma ou le code source dans le cadre d'une récupération de routine. Ne supprimez pas `data/bpm.db` pour que le démarrage réussisse.

5. Choisissez le plus petit chemin de réexécution et d'escalade.

Réexécution ciblée : pour la fumée de produit, uniquement si la sortie de documentation est impliquée et uniquement pour les contrats de documentation.

Avertissement :

Si les sondes restent inaccessibles après les vérifications de configuration et de processus, arrêtez les exécutions d'intégration automatisées et faites remonter les commandes, les journaux, les noms de configuration et les preuves de sonde.

L'opérateur peut distinguer un processus défaillant, une sonde inaccessible et une limite de déploiement sans muter les données utilisateur.

Une fois les sondes réussies, revenez à la rubrique Démarrage contrôlé par l'état ou Exemples d'API réutilisables avant d'exécuter des intégrations en mutation.

Résoudre les erreurs de cache de schéma et de validation d'API

Diagnostiquez le cache de schéma indisponible, le canal de schéma inconnu et les échecs de validation des politiques Firefox sans corriger les artefacts de schéma générés.

Utilisez cette procédure lorsque `POST $BPM_BASE_URL/api/validate/release-153`, `POST $BPM_BASE_URL/api/validate/beta-999` ou une validation destinée à l'utilisateur renvoie 400, 404, 422 ou 503 ; un document non objet peut renvoyer 200 `ok=false` avec `Expected object with policy mappings`. La validation

La validation BPM dépend des canaux de schéma maintenus et de la configuration du schéma/cache local. Le diagnostic enregistre si le problème concerne la forme de la demande, un profil inconnu, des données de stratégie Firefox non valides ou un cache de schéma indisponible.

1. Enregistrez la demande de validation et la réponse exactes.

Stocke le hachage du corps de la requête nettoyé, le canal de schéma, Statut HTTP , detail, message, error, et issues. Gardez le candidat `policies.json` en dehors des journaux s'il contient des valeurs client.

2. Classifiez d'abord les échecs de forme de demande.

Un document non-objet ou un document `policies` manquant est une entrée de l'opérateur, pas un défaut du cache de schéma ; Le BPM actuel peut signaler cela comme 200 `ok=false`. Réexécution ciblée : `POST $BPM_BASE_URL/api/validate/release-153` avec `{"document":{"policies":{"DisableTelemetry":true}}}`.

3. Classifier les échecs de canal de schéma et de cache.

Un profil tel que `beta-999` peut renvoyer 404 ; un schéma enregistré indisponible peut renvoyer 503. Vérifiez `BPM_SCHEMA_BASE_URL`, `BPM_SCHEMA_CACHE_DIR` et `BPM_SCHEMA_HTTP_TIMEOUT` avant toute reconstruction plus large.

4. Utiliser des contrôles d'actualisation et de validation maintenus.

Réexécution ciblée : pour la dérive de la documentation de l'API ou les tests de schéma de produit lorsque le comportement de l'API a changé. Ne modifiez pas manuellement `app/schemas/cache` ni les squelettes Firefox générés.

5. Arrêtez la récupération dangereuse et préservez les preuves.

Si la même demande valide change de statut au fil des exécutions, conservez les corps de réponse, la révision source, le chemin du cache et les variables d'environnement.

Avertissement :

Ne modifiez pas les artefacts de schéma générés ou les lignes de base de données dans le cadre d'une récupération de routine ; séparer les défauts du produit des erreurs de charge utile de l'opérateur.

L'opérateur a un échec de validation classifié avec une plus petite réexécution et aucune mutation de schéma cachée.

Une fois la validation réussie, revenez aux processus de validation avant application ou d'importation-révision-exportation.

Résoudre les échecs d'importation et d'exportation de Firefox

Diagnostiquez les échecs d'importation JSON, d'importation en plusieurs parties et d'exportation de politiques Firefox tout en préservant la source et les preuves exportées.

Utilisez cette procédure lorsque POST `$BPM_BASE_URL/api/profiles/import/firefox/policies.json` ou GET `$BPM_BASE_URL/api/export/profiles/42/firefox/policies.json` renvoie 400, 404, 409, 415 ou 422.

Le dépannage d'importation/exportation sépare la validité du document source, la forme des champs en plusieurs parties, les noms de profil en double, l'accès aux profils archivés et l'exportation canonique de Firefox des outils de déploiement en aval.

1. Identifiez l'opération et le type de contenu.

Enregistrez si la requête était JSON `FirefoxPoliciesJsonImportRequest`, `multipart/form-data` avec `file=@policies.json` ou exportez avec `download=1`, `pretty=1` ou `include_deleted=true`.

2. Validez le document source Firefox avant de réessayer l'importation.

Réexécution ciblée : POST `$BPM_BASE_URL/api/validate/release-153` avec le même document.`policies`. Si la validation échoue, corrigez le document source en dehors de BPM avant une autre importation.

3. Vérifiez les erreurs de l'opérateur spécifiques à l'importation.

Pour 409, choisissez un nouveau nom de profil synthétique ou archivez/restaurez intentionnellement. Pour 415, vérifiez les champs `application/json` ou de téléchargement partitionné. Pour 422, enregistrez les erreurs au niveau du champ et ne réécrivez pas directement le stockage du profil.

4. Vérifiez l'état spécifique à l'exportation.

Pour 404, relisez GET `$BPM_BASE_URL/api/profiles/42?include_deleted=true`. Pour les profils archivés, décidez si `include_deleted=true` est approprié. Comparez le hachage du document exporté avec le hachage source examiné ; Les métadonnées du profil BPM et les métadonnées de conformité ne sont pas exportées.

5. Enregistrer les conditions de récupération et d'arrêt.

Réexécution ciblée : pour des exemples documentés.

Avertissement :

Ne réessayez pas aveuglément après des preuves d'importation/exportation incompatibles ; stocker le hachage source, le hachage d'exportation, le profil ID, la révision, le statut HTTP et la décision de l'opérateur.

L'opérateur peut identifier si l'échec appartient au JSON source, aux champs en plusieurs parties, à l'état du cycle de vie du profil ou aux preuves d'exportation.

Après la récupération, joignez les preuves au dossier d'audit du produit de contrôle externe.

Résoudre les problèmes de base de données et de stockage

Diagnostiquez les problèmes locaux de SQLite, de chemin de stockage, de sauvegarde et de visibilité du profil sans modifier directement les lignes de la base de données.

Utilisez cette procédure lorsque les profils disparaissent, que les écritures échouent, que les preuves de migration ne sont pas claires, que les sommes de contrôle de sauvegarde ne correspondent pas ou que `BPM_DATABASE_URL` pointe vers un emplacement inattendu.

Le déploiement source actuel utilise le stockage local tel que `sqlite+aiosqlite:///./data/bpm.db` et les fichiers appartenant au référentiel sous `data/`, les chemins de cache de schéma et les chemins du site de documentation. Il n'existe aucune orchestration gérée de sauvegarde ou de restauration dans BPM.

1. Confirmez l'URL de la base de données active et le répertoire de travail.

Enregistrez `BPM_DATABASE_URL`, le répertoire actuel, la révision source et si le processus s'exécute à partir du système de fichiers Linux ou de `/mnt/c`. Ne présumez pas que deux obus pointent vers le même `data/bpm.db`.

2. Vérifiez les preuves de migration et d'API de profil avant d'ouvrir les fichiers de stockage.

Réexécution ciblée : `alembic upgrade head`, puis `curl -fsS http://127.0.0.1:8000/api/profiles?limit=50`. Si l'API renvoie les profils attendus, le symptôme peut être un filtre, un cycle de vie ou l'état du navigateur.

3. Vérifiez les preuves de sauvegarde avant la récupération destructive.

Vérifiez les enregistrements `mkdir -p backups`, `cp data/bpm.db backups/bpm-pre-update.db` et `sha256sum backups/bpm-pre-update.db`. Conservez les exportations des profils `policies.json` critiques comme preuve distincte.

4. Évitez les modifications directes des lignes de la base de données.

Ne modifiez pas les lignes SQLite, les artefacts générés ou les fichiers de cache de schéma en tant que récupération de routine. Utilisez plutôt des procédures documentées d'archivage, de restauration, d'importation, d'exportation ou de mise à jour de profil à partir de la source.

5. Transférez les preuves de stockage lorsque la récupération n'est pas sécurisée.

Inclut l'URL de la base de données, le chemin du fichier, la somme de contrôle, la commande de migration, la réponse de l'API et les preuves de sauvegarde/exportation.

Avertissement :

S'il n'existe aucune sauvegarde vérifiée avant panne, arrêtez avant de supprimer ou de remplacer `data/bpm.db` ; BPM ne fournit pas de récupération à un moment donné.

L'opérateur dispose d'un diagnostic de stockage qui préserve les données utilisateur et sépare l'état de la base de données de UI ou des symptômes de filtre.

Une fois le stockage stable, réexécutez l'état/préparation et le plus petit processus d'API affecté.

Dépanner le réseau WSL et les dépendances obsolètes

Diagnostiquez Windows 10/11 via les problèmes d'accès WSL, le placement du système de fichiers Linux et les dépendances Python obsolètes après les modifications de source.

Utilisez cette procédure lorsque Windows ne peut pas atteindre un processus BPM hébergé par WSL, que les sondes fonctionnent dans WSL mais pas à partir de Windows, que les performances des fichiers sont instables sous `/mnt/c` ou que les dépendances semblent obsolètes après `git checkout <approved-new-ref>`.

Le dépannage WSL sépare l'accès au navigateur Windows, la liaison des processus Linux, le placement du système de fichiers et l'actualisation de l'environnement Python. BPM ne fournit actuellement ni service Windows natif ni programme d'installation Windows.

1. Confirmez l'emplacement de l'extraction source et de l'environnement virtuel.

Préférez le système de fichiers Linux pour l'extraction et `.venv`. Enregistrez si les fichiers se trouvent sous `/mnt/c`, ce qui peut modifier les performances et les autorisations. Ne mélangez pas plusieurs vérifications pendant le diagnostic.

2. Vérifiez la liaison du processus WSL et l'itinéraire Windows.

Vérifiez `BPM_HOST="0.0.0.0"`, `BPM_PORT="8000"`, `curl -fsS http://127.0.0.1:8000/health` dans WSL et l'accès au navigateur depuis Windows. Si Windows échoue, enregistrez les preuves du pare-feu et de la redirection de port.

3. Actualiser les dépendances après un mouvement de source ou des modifications de révision.

Réexécution ciblée : `source .venv/bin/activate`, `python -m pip install --upgrade pip` et `pip install -e ".[dev]"`. Si les erreurs de dépendance persistent, enregistrez les noms des packages et la révision source plutôt que de modifier manuellement les packages installés.

4. Séparez les symptômes de dépendance obsolètes des problèmes de schéma ou de base de données.

Si l'importation/exportation ou la validation échoue après l'actualisation des dépendances, utilisez le schéma/cache et le dépannage de l'importation/exportation. Si la visibilité du profil change, utilisez le dépannage de la base de données/stockage.

5. Arrêtez les hypothèses dangereuses de Windows natif.

Ne créez pas de services Windows non documentés, ne déplacez pas la base de données entre les systèmes de fichiers et ne revendiquez pas le comportement du programme d'installation natif.

Avertissement :

Si la mise en réseau WSL reste ambiguë, arrêtez les intégrations externes et faites remonter la distribution WSL, le chemin d'extraction, l'adresse de liaison, l'URL Windows et les sorties de sonde.

L'opérateur peut distinguer les défauts de routage WSL, de placement du système de fichiers et d'actualisation des dépendances sans modifier le stockage du produit.

Une fois l'accès WSL stable, revenez à la vérification du déploiement source et au démarrage sécurisé.

Dépanner un portail de documentation indisponible

Collectez l'état visible de /help/ et les éléments de déploiement lorsque le portail de documentation installé est indisponible.

Utilisez cette procédure lorsque /help/ signale un état absent, obsolète, incompatible, incomplet ou introuvable.

Le portail de documentation fait partie de l'interface BPM. Sa disponibilité dépend de la correspondance entre l'artefact installé et la version BPM en cours d'exécution.

1. Enregistrer l'URL du portail et l'état visible

Le portail de documentation fait partie de l'interface BPM. Sa disponibilité dépend de la correspondance entre l'artefact installé et la version BPM en cours d'exécution.

2. Confirmer l'instance BPM en cours et le paramètre de documentation

Le portail de documentation fait partie de l'interface BPM. Sa disponibilité dépend de la correspondance entre l'artefact installé et la version BPM en cours d'exécution.

3. Arrêter les actions dépendantes de la documentation

Avertissement :

Ne considérez pas un portail absent ou incompatible comme un système d'aide opérationnel et ne modifiez pas manuellement les fichiers installés.

4. Transmettre les éléments au responsable du déploiement BPM

Le portail de documentation fait partie de l'interface BPM. Sa disponibilité dépend de la correspondance entre l'artefact installé et la version BPM en cours d'exécution.

5. Vérifier le portail après la récupération

Le portail de documentation fait partie de l'interface BPM. Sa disponibilité dépend de la correspondance entre l'artefact installé et la version BPM en cours d'exécution.

Le portail de documentation fait partie de l'interface BPM. Sa disponibilité dépend de la correspondance entre l'artefact installé et la version BPM en cours d'exécution.

Vérifier le portail après la récupération.

Examiner les questions sur l'exposition du réseau et la préparation au proxy

Préparez des notes d'exposition réseau et des questions de proxy inverse sans publier des recettes TLS ou proxy non prises en charge en tant que comportement BPM.

L'évaluation actuelle de l'état de préparation de la source à nœud unique est terminée et l'opérateur sait si les utilisateurs atteindront BPM via un bouclage, une adresse transférée WSL, un hôte de laboratoire ou un proxy externe.

BPM peut s'exécuter sur un hôte et un port configurés, mais la documentation BPM ne fournit pas de proxy inverse officiel, de terminaison TLS, de passerelle d'authentification, de stratégie d'en-tête ou de recette de limitation de débit.

1. Choisissez la plus petite exposition réseau qui satisfait à l'évaluation actuelle.

Préférez 127.0.0.1. Utilisez `BPM_HOST="0.0.0.0"` uniquement lorsque le transfert WSL ou un réseau de laboratoire contrôlé l'exige, et enregistrez pourquoi `BPM_PORT="8000"` est accessible.

2. Notez les questions de préparation au proxy avant d'ajouter un proxy externe.

Indiquez qui possède les certificats TLS, le routage du nom d'hôte, les limites de taille des requêtes, les valeurs de délai d'attente, les origines CORS via `BPM_CORS_ALLOW_ORIGINS`, les en-têtes transférés, les journaux d'accès et la réponse aux incidents. Ces questions peuvent être préparées dès maintenant ; les recettes officielles TLS/proxy sont prévues mais pas encore mises en œuvre.

3. Vérifiez que les sondes fonctionnent toujours à travers la limite choisie.

```
curl -fsS http://127.0.0.1:8000/health
curl -fsS http://127.0.0.1:8000/health/ready
```

Si un proxy externe est utilisé dans un laboratoire, enregistrez à la fois les sondes source directe et les sondes côté proxy. Ne traitez pas une réponse du proxy 200 comme une preuve d'authentification, d'autorisation ou de HA.

4. Marquer explicitement les revendications de réseau non prises en charge.

Avertissement :

BPM ne fournit pas de services packagés, de recettes officielles TLS/proxy, de clustering HA pris en charge, de règles WAF, de signature de requêtes, de limitation de débit, SSO, OAuth/OIDC, de contrôle d'accès basé sur les rôles ou de renforcement de la production. Conservez-les comme contrôles externes ou exigences différées.

La note réseau sépare ce qui peut être configuré maintenant des décisions de proxy inverse et TLS qui restent externes ou différées.

Utilisez la rubrique de surveillance, de sauvegarde et de fenêtre de mise à jour avant le démarrage d'une fenêtre d'évaluation partagée.

Planifier les entrées de surveillance, les preuves de sauvegarde et les fenêtres de mise à jour

Préparez des preuves opérationnelles de l'état actuel pour une instance exécutée à la source tout en évitant les réclamations concernant la surveillance gérée, l'automatisation officielle de la restauration ou les mises à niveau propagées.

L'instance a réussi l'examen de préparation des sources et d'exposition du réseau, et un seul administrateur ou propriétaire DevOps est responsable de la fenêtre d'évaluation.

Jusqu'à ce que les décisions relatives aux distributions et aux services d'exécution soient prises, la planification des opérations doit se concentrer sur les entrées observables, les preuves de sauvegarde et d'exportation manuelles, ainsi que les fenêtres de mise à jour explicites pour un nœud source.

1. Répertoirez les entrées de surveillance qui existent aujourd'hui.

Enregistrez l'état du processus, la sortie du terminal de premier plan, stdout, stderr, GET /health, GET /health/ready, les exemples de résultats d'API sélectionnés, l'accès à la documentation via /help/ et la révision source de git rev-parse --short HEAD.

2. Collectez les preuves de sauvegarde et d'exportation avant la fenêtre.

```
mkdir -p backups
cp data/bpm.db backups/bpm-pre-update.db
sha256sum backups/bpm-pre-update.db
```

Exportez également des documents Firefox représentatifs tels que GET \$BPM_BASE_URL/api/export/profiles/42/firefox/policies.json et stockez les preuves policies.json obtenues avec des sommes de contrôle.

3. Planifiez une fenêtre d'arrêt pour les mises à jour du code source.

Prévoyez l'arrêt du nœud avant git fetch --tags, l'actualisation des dépendances, alembic upgrade head, et . Les mises à niveau progressives ne sont pas prises en charge.

4. Enregistrez les lacunes comme des bloqueurs et non comme des hypothèses cachées.

Avertissement :

BPM ne fournit pas de surveillance gérée, de planification de sauvegarde automatisée, d'automatisation de restauration officielle, de récupération à un moment précis, de secrets gérés, de rotation de jetons ou d'orchestration de mise à niveau continue. Utilisez des preuves manuelles et des conditions d'arrêt du runbook de mise à jour.

Le plan d'opération contient des entrées de surveillance, des preuves de sauvegarde/exportation et une fenêtre de mise à jour réaliste pour l'opération source actuelle.

Joignez ces preuves au HA et à l'enregistrement à limites différées afin que les travaux de distribution ultérieurs puissent réutiliser la même liste de contrôle.

Assistant de documentation BPM

Utiliser l'assistant de documentation dans BPM 0.9.3

Vérifiez la limite publiée de l'avis d'entraînement sans rendre BPM ou la recherche déterministe dépendants du chat.

Utilisez un checkout BPM 0.9.3 pris en charge et les ressources de base décrites dans [Configuration système minimale pour BPM](#).

Dans cette version, le chat est uniquement une interface de lecture localisée. Il ne démarre ni n'installe de modèle, ne récupère pas la documentation, n'utilise pas d'index E5, ne génère pas de réponse, n'affiche pas de citations et ne contacte pas de sources externes. La recherche déterministe reste distincte et disponible.

1. Démarrez BPM avec le flux depuis les sources normal et pris en charge.

N'ajoutez pas de configuration de modèle, d'index, d'embeddings ou de fournisseur externe au démarrage de BPM 0.9.3.

2. Ouvrez le portail de documentation dans la langue du lecteur.

Vérifiez que l'assistant s'ouvre en bas à droite et que les contrôles de recherche ordinaires restent inchangés.

3. Envoyez une question sur BPM.

La transcription doit afficher l'avis localisé indiquant que le modèle local est en cours d'entraînement et que la fonction sera disponible dans une version ultérieure.

4. Envoyez une question hors sujet.

Le même avis est attendu ; le chat n'est pas un assistant généraliste dans cette version.

5. Trouvez la réponse actuelle avec la recherche déterministe dans la documentation.

La recherche, les filtres, la navigation et l'ordre des résultats restent le chemin pris en charge pour les informations produit.

L'assistant affiche l'avis d'entraînement localisé publié, tandis que BPM et la recherche déterministe restent utilisables indépendamment.

N'installez pas de modèle et ne créez pas de génération RAG pour 0.9.3. Signalez comme défaut de documentation tout écart d'avis, de langue ou de limite avec la recherche ordinaire.

Maintenir la limite de l'assistant de documentation dans BPM 0.9.3

Conservez l'exactitude de l'avis d'entraînement localisé et de la limite avec la recherche déterministe lors de la maintenance de la documentation BPM.

Préparez la révision des sources BPM approuvée et le flux de publication de la documentation.

La version 0.9.3 ne contient aucun modèle local, artefact de modèle, génération RAG, magasin d'embeddings, magasin de citations ou fournisseur de sources externes à maintenir. Ne créez pas d'état de maintenance pour des fonctions non livrées dans cette version.

1. Appliquez la révision BPM approuvée et régénérez la documentation.

Utilisez le flux documentaire normal, y compris `make docs-install-dev` pour préparer un environnement de développement.

2. Ouvrez la documentation publiée dans chaque langue prise en charge.

Vérifiez l'entrée de l'assistant et le fonctionnement indépendant de la recherche déterministe.

3. Envoyez une question dans chaque langue.

Vérifiez l'avis propre à la langue et l'absence de réponse, source, citation, affirmation externe ou installation de modèle.

4. N'exécutez aucune commande d'installation de modèle ou de génération RAG pour 0.9.3.

Elles appartiennent à une version ultérieure et ne constituent pas une récupération de l'avis d'entraînement.

5. Consignez la révision de la documentation et le résultat de la vérification.

Traitez tout écart entre l'avis publié et la limite de recherche déterministe comme un défaut de documentation.

Le portail publié conserve un avis localisé dans le chat et un chemin distinct vers les informations actuelles grâce à la recherche déterministe.

Ne conservez pas les questions des lecteurs comme artefact de maintenance. Réévaluez les opérations de modèle et de récupération uniquement lorsqu'une version ultérieure les livre explicitement.